Sensor Porting Introduction

ISP 6s

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

驱动常见问题

- 少Feature control
- Winsize 错误
- 少Mipi Pixel rate
- Sensor mode个数错误

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

FYR

Guide	FAQ17467	[Camera Drv]mtk online 下载camera driver使用说明
	FAQ20596	image sensor driver template change for isp5.0
	FAQ22915	image sensor driver template change for ISP6s
	FAQ22441	Porting Camera Sensor 的时候新增metadata与tuning files的方法
	FAQ21141	Flicker Parameters Generation with isp binning
Debug	FAQ21124	how to check mipi_pixel_rate of image sensor driver
	FAQ22328	Camera Sensor Driver Check
	FAQ22667	ERR_Sensor_Data_Mismatch与ERR_Sensor_No_Signal的初步分析方法
	FAQ22426	Sensor info exception CRDISPATCH_KEY:Sensor_Info_preCheck的解决方法
	FAQ23393	CRDISPATCH_KEY:CCU -> MSG_TO_APMCU_CCU_ASSERT:240
	FAQ22940	Sensor driver引起Camera Performance问题的排查和优化方法
Customized	FAQ22847	Android Q 新增一组Regulator的方法
	FAQ22659	[Android Q] Camera Sensor Regulator Restricting voltage 的处理方法

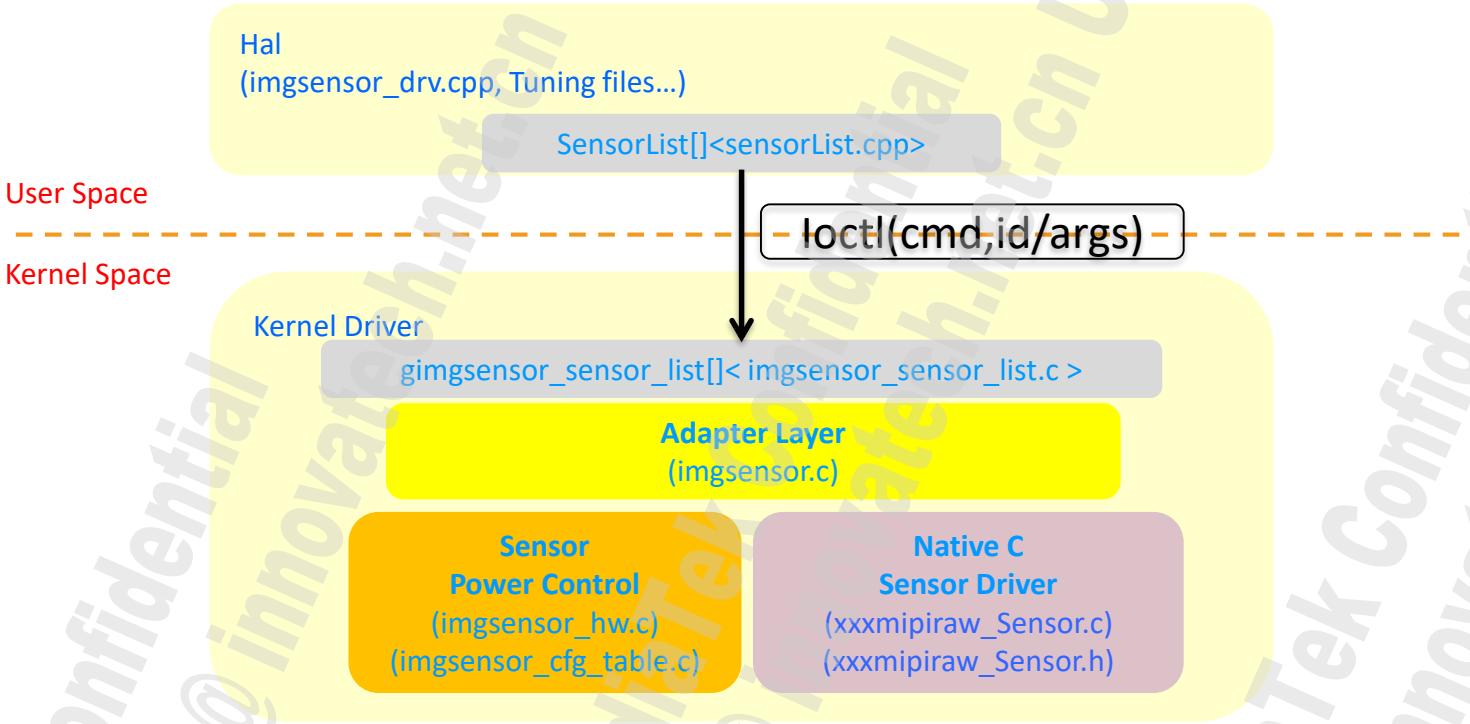
What's Changed

- Auto deviceinfo generate
  - Function
  - Parameter
- Some new function
- Sensor_Info_preCheck
- Metadata
- Vblanking

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

Driver Architecture



- Imgsensor Drv
  - User space driver
- Adapter layer
  - A adapter layer for Linux character device driver and native sensor driver
- Sensor power control
  - Control the sensor power on/off

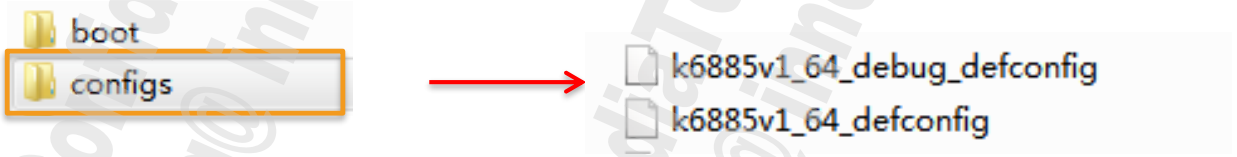
Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

Sensor Driver Porting

- Config Files
  - /device/[mediatekprojects|mediateksample]/{project}/

  - /kernel-4.14/arch/arm64/configs/



  - \device\mediatek\common\

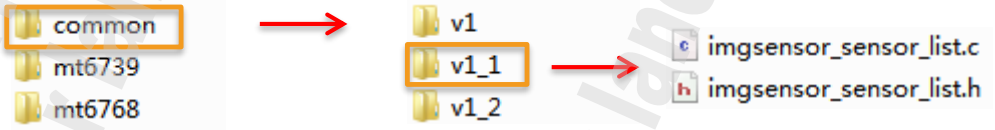
Sensor Driver Porting

- Kernel Driver
  - /kernel-4.14/drivers/misc/mediatek/imgsensor/src/common/v1_1/

  - /kernel-4.14/drivers/misc/mediatek/imgsensor/src/{platform}/

  - /kernel-4.14/drivers/misc/mediatek/imgsensor/·

  - \kernel-4.14\drivers\misc\mediatek\imgsensor\src\



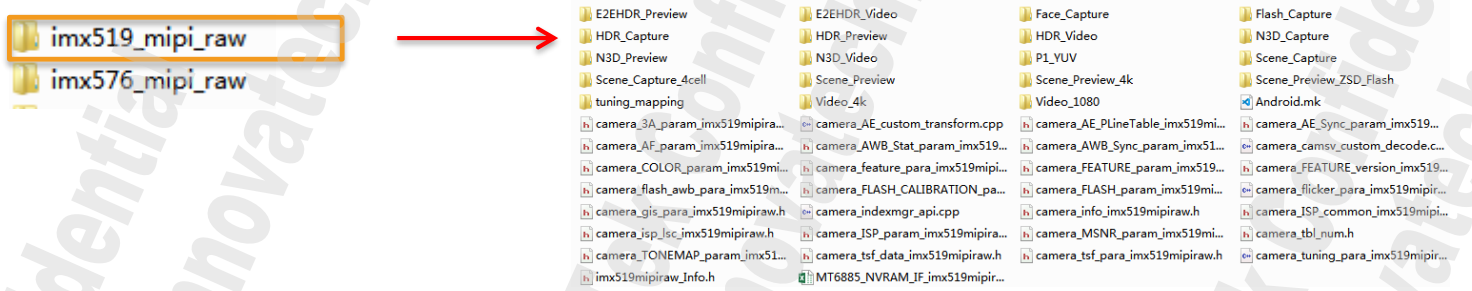
Sensor Driver Porting

- Kernel Driver
  - /kernel-4.14/drivers/misc/mediatek/dws/{platform}/

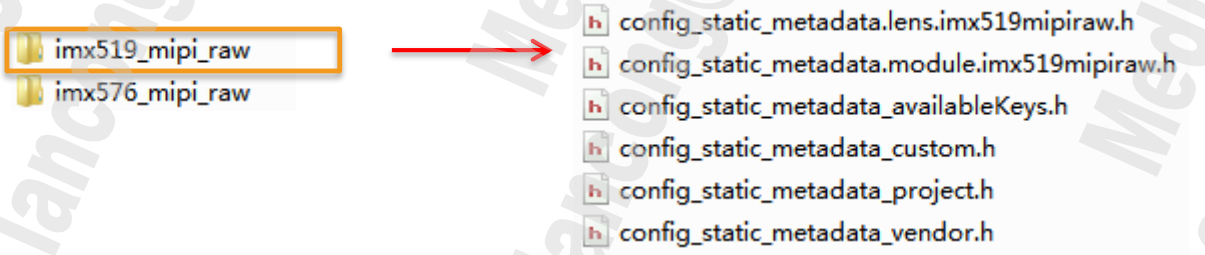
  - \kernel-4.14\arch\arm64\boot\dts\

Sensor Driver Porting

- Hal Driver
  - /vendor/mediatek/proprietary/custom/ ${platform}/hal/imgsensor\ver1



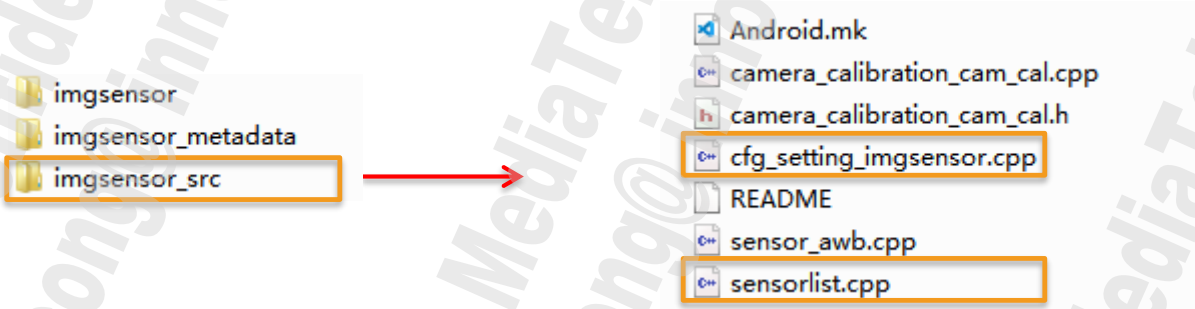
  - /vendor/mediatek/proprietary/custom/ ${platform}/hal/imgsensor_metadata



Sensor Driver Porting

- Hal Driver
  - /vendor/mediatek/proprietary/custom/common/hal/imgsensor_metadata/sensor

  - /vendor/mediatek/proprietary/custom/mt6885/hal/



  - vendor\mediatek\proprietary\hardware\mtkcam\include\mtkcam\

How To Add A New Sensor

- Step1

/device/[mediatekprojects|mediateksample]/{project}/**ProjectConfig.mk**

/kernel-4.14/arch/arm64/configs/**${project}_debug_defconfig**

/kernel-4.14/arch/arm64/configs/**${project}_defconfig**

1.1、ProjectConfig.mk 如下修改

(a) 修改imgsensor相关

eg: main xxx_mipi_raw, sub xxxx_mipi_raw, main2 xxxx_mipi_raw

- CUSTOM_HAL_IMGSENSOR = **xxxx_mipi_raw xxxx_mipi_raw xxxx_mipi_raw**
- CUSTOM_KERNEL_IMGSENSOR = **xxxx_mipi_raw xxxx_mipi_raw xxxx_mipi_raw**
- ~~CUSTOM_HAL_MAIN_IMGSENSOR = **xxxx_mipi_raw**~~
- ~~CUSTOM_HAL_SUB_IMGSENSOR = **xxxx_mipi_raw**~~
- ~~CUSTOM_KERNEL_MAIN_IMGSENSOR = **xxxx_mipi_raw**~~
- ~~CUSTOM_KERNEL_SUB_IMGSENSOR = **xxxx_mipi_raw**~~
- ~~CUSTOM_HAL_MAIN2_IMGSENSOR = **xxxx_mipi_raw**~~
- ~~CUSTOM_KERNEL_MAIN2_IMGSENSOR = **xxxx_mipi_raw**~~
- MTK_CAM_MAX_NUMBER_OF_CAMERA = 5 → 当前项目需要支持的camera的个数

How To Add A New Sensor

(b) 修改lens相关

sensor porting时，先将lens配置为dummy。

- CUSTOM_HAL_LENS = **dummy_lens**
- CUSTOM_KERNEL_LENS = **dummy_lens**
- CUSTOM_HAL_MAIN_LENS = **dummy_lens**
- CUSTOM_HAL_SUB_LENS = **dummy_lens**
- CUSTOM_KERNEL_MAIN_LENS = **dummy_lens**
- CUSTOM_KERNEL_SUB_LENS = **dummy_lens**

(c) 修改flashlight相关

支持Flashlight设置为constant_flashlight, 不支持设置为dummy_flashlight

- CUSTOM_HAL_FLASHLIGHT = dummy_flashlight
- CUSTOM_KERNEL_FLASHLIGHT = dummy_flashlight

1.2 ${project}_debug_defconfig 与/${project}_defconfig

CONFIG_CUSTOM_KERNEL_IMGSENSOR="**xxxx**_mipi_raw **xxxx**_mipi_raw "

CONFIG_MTK_CAM_CAL=n　　先配置为n不配置eeprom

How To Add A New Sensor

- Step2

1)/kernel-4.14/drivers/misc/mediatek/imgsensor/inc/kd_imgsensor.h

2)/device/mediatek/common/kernel-headers/kd_imgsensor.h

- Add new sensor ID define ,eg:

```
#define IMX519_SENSOR_ID                    0x0519
```

The value of Sensor ID learn from specifc sensor datasheet

- Add new sensor name define,eg:

```
#define SENSOR_DRVNAME_IMX519_MIPI_RAW          "imx519_mipi_raw"
```

How To Add A New Sensor

- Step2

3)vendor/mediatek/proprietary/hardware/mtkcam/include/mtkcam/drv/IHalSensor.h

```
enum
{
    SENSOR_DEV_NONE = 0x00,
    SENSOR_DEV_MAIN = 0x01,
    SENSOR_DEV_SUB  = 0x02,
    SENSOR_DEV_PIP  = 0x03,
    SENSOR_DEV_MAIN_2 = 0x04,
    SENSOR_DEV_MAIN_3D = 0x05,
    SENSOR_DEV_SUB_2 = 0x08,
    SENSOR_DEV_MAIN_3 = 0x10,
    SENSOR_DEV_SUB_3 = 0x20,
    SENSOR_DEV_MAIN_4 = 0x40
};

enum
{
    IDX_MAIN_CAM        = 0x00,
    IDX_SUB_CAM         = 0x01,
    IDX_MAIN2_CAM       = 0x02,
    IDX_SUB2_CAM        = 0x03,
    IDX_MAIN3_CAM       = 0x04,
    IDX_SUB3_CAM        = 0x05,
    IDX_MAIN4_CAM       = 0x06,
    IDX_MAX_CAM_NUMBER,
};
```

根据需求增加：
SENSOR_DEV_XXXX 和 IDX_XXX_CAM

How To Add A New Sensor

- Step3

/kernel-4.14/drivers/misc/mediatek/imgsensor/src/common/v1_1/imgsensor_sensor_list.h

- Add new sensor init function declaration, eg:

```
UINT32 IMX519_MIPI_RAW_SensorInit(struct SENSOR_FUNCTION_STRUCT **pfFunc);
```

/kernel-4.14/drivers/misc/mediatek/imgsensor/src/common/v1_1/imgsensor_sensor_list.c

- Add new sensor in kernel gimgsensor_sensor_list[],eg:

```
#if defined(IMX519_MIPI_RAW)
{IMX519_SENSOR_ID, SENSOR_DRVNAME_IMX519_MIPI_RAW, IMX519_MIPI_RAW_SensorInit},
#endif
```

Sensor ID

Sensor name

Sensor init Funciton

How To Add A New Sensor

- Step4

  /vendor/mediatek/proprietary/custom/${project}/hal/imgsensor_src/sensorlist.cpp

  Add new sensor in hal SensorList[]

```
#if defined(IMX519_MIPI_RAW)
    RAW_INFO_M(IMX519_SENSOR_ID, DEFAULT_MODULE_INDEX, DEFAULT_MODULE_ID, SENSOR_DRVNAME_IMX519_MIPI_RAW, CAM_CALGetCalData),
#endif
```

Note: sensorlist.cpp中的SensorList[]与imgsensor_sensor_list.c中的gimgsensor_sensor_list的sensor的顺序必须一致，否则user space 和kernel space在通过ioctl传递命令id时会对应错误。

How To Add A New Sensor

- Step5

/kernel-4.14/drivers/misc/mediatek/imgsensor/src/ /${platform} /camera_hw/imgsensor_cfg_table.c

确认上电引脚是GPIO供电还是PMIC供电：

I2C_DEV保持默认不用修改

```
struct IMGSENSOR_HW_CFG imgsensor_custom_config[] = {
    {
        IMGSENSOR_SENSOR_IDX_MAIN,
        IMGSENSOR_I2C_DEV_0,
        {
            {IMGSENSOR_HW_PIN_MCLK,   IMGSENSOR_HW_ID_MCLK},
            {IMGSENSOR_HW_PIN_AVDD,   IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_DOVDD,  IMGSENSOR_HW_ID_REGULATOR},
            {IMGSENSOR_HW_PIN_DVDD,   IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_PDN,    IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_RST,    IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_NONE,   IMGSENSOR_HW_ID_NONE},
        },
    },
    {
        IMGSENSOR_SENSOR_IDX_SUB,
        IMGSENSOR_I2C_DEV_1,
        {
            {IMGSENSOR_HW_PIN_MCLK,   IMGSENSOR_HW_ID_MCLK},
            {IMGSENSOR_HW_PIN_AVDD,   IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_DOVDD,  IMGSENSOR_HW_ID_REGULATOR},
            {IMGSENSOR_HW_PIN_DVDD,   IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_PDN,    IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_RST,    IMGSENSOR_HW_ID_GPIO},
            {IMGSENSOR_HW_PIN_NONE,   IMGSENSOR_HW_ID_NONE},
        },
    },
```

How To Add A New Sensor

- Step5

/kernel-4.14/drivers/misc/mediatek/imgsensor/src/ /${platform} /camera_hw/imgsensor_cfg_table.c

若需要MIPI_SWITCH请添加define

```
struct IMGSENSOR_HW_POWER_SEQ platform_power_sequence[] = {
#ifdef MIPI_SWITCH
    {
        PLATFORM_POWER_SEQ_NAME,
        {
            {
                IMGSENSOR_HW_PIN_MIPI_SWITCH_EN,
                IMGSENSOR_HW_PIN_STATE_LEVEL_0,
                0,
                IMGSENSOR_HW_PIN_STATE_LEVEL_HIGH,
                0
            },
            {
                IMGSENSOR_HW_PIN_MIPI_SWITCH_SEL,
                IMGSENSOR_HW_PIN_STATE_LEVEL_HIGH,
                0,
                IMGSENSOR_HW_PIN_STATE_LEVEL_0,
                0
            },
        },
        IMGSENSOR_SENSOR_IDX_SUB,
    },
```

EN管脚

上电拉低

上电延时

下电延时

下电拉高

SEL管脚

上电拉高

上电延时

下电拉低

下电延时

操作sub sensor

```
struct IMGSENSOR_HW_POWER_INFO {
    enum IMGSENSOR_HW_PIN pin;
    enum IMGSENSOR_HW_PIN_STATE pin_state_on;
    u32 pin_on_delay;
    enum IMGSENSOR_HW_PIN_STATE pin_state_off;
    u32 pin_off_delay;
};
```

How To Add A New Sensor

- Step5

/kernel-4.14/drivers/misc/mediatek/imgsensor/src/ ${platform}/camera_hw/imgsensor_cfg_table.c

As for Power On/Off Sequence ,please refer to specific sensor datasheet;

```
#if defined(IMX519_MIPI_RAW)
        {
                SENSOR_DRVNAME_IMX519_MIPI_RAW,
                {
                        {PDN, Vol_Low, 0},
                        {RST, Vol_Low, 0},
                        {AVDD, Vol_2800, 0},
                        {AFVDD, Vol_2800, 0},
                        {DVDD, Vol_1200, 0},
                        {DOVDD, Vol_1800, 1},
                        {SensorMCLK, Vol_High, 5},
                        {PDN, Vol_High, 0},
                        {RST, Vol_High, 1}
                },
        },
```

PowerType

Voltage

Delay Time in ms

上电顺序

How To Add A New Sensor

- Step6

/kernel-4.14/arch/arm64/boot/dts/mediatek/${project}.dts

将原本的cust_mt6885_camera.dtsi文件复制一份重命名为cust_{project}_camera.dtsi
在k6885v1_64.dts的最后会有 #include “cust_mt6885_camera.dtsi”
将其修改为#include “cust_{project}_camera.dtsi”，然后修改cust_{project}_camera.dtsi 中pio节点的各个Pin Number
注意：可以先删除dts中eeprom的配置，若eeprom的dts配置错误可能会导致不预期的问题发生。

Eg：

```
&pio {
    camera_pins_cam0_rst_0: cam0@0 {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO144__FUNC_GPIO144>;
            slew-rate = <1>;
            output-low;
        };
    };
    camera_pins_cam0_rst_1: cam0@1 {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO144__FUNC_GPIO144>;
            slew-rate = <1>;
            output-high;
        };
    };
```

```
    camera_pins_cam0_mclk_off: camera_pins_cam0_mclk_off {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO150__FUNC_GPIO150>;
            drive-strength = <1>;
        };
    };
    camera_pins_cam0_mclk_2ma: camera_pins_cam0_mclk_2ma {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO150__FUNC_CMMCLK1>;
            drive-strength = <0>;
        };
    };
    camera_pins_cam0_mclk_4ma: camera_pins_cam0_mclk_4ma {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO150__FUNC_CMMCLK1>;
            drive-strength = <1>;
        };
    };
```

How To Add A New Sensor

- Step6

在cust_{project}_camera.dtsi文件中kd_camera_hw1节点中将采用GPIO供电方式的pin增加如下子节点，如下图：

```
&kd_camera_hw1 {
    pinctrl-names = "default",
            "cam0_rst0", "cam0_rst1",
            "cam0_pnd0", "cam0_pnd1",
            "cam1_rst0", "cam1_rst1",
            "cam1_pnd0", "cam1_pnd1",
            "cam2_rst0", "cam2_rst1",
            "cam2_pnd0", "cam2_pnd1",
            "cam4_rst0", "cam4_rst1",
            "cam4_pnd0", "cam4_pnd1",
            "cam0_ldo_vcamd_0", "cam0_ldo_vcamd_1",
            "cam0_ldo_vcama_0", "cam0_ldo_vcama_1",
            "cam1_ldo_vcamd_0", "cam1_ldo_vcamd_1",
            "cam1_ldo_vcama_0", "cam1_ldo_vcama_1",

    pinctrl-17 = <&camera_pins_cam0_vcamd_0>;
    pinctrl-18 = <&camera_pins_cam0_vcamd_1>;
    pinctrl-19 = <&camera_pins_cam0_vcama_0>;
    pinctrl-20 = <&camera_pins_cam0_vcama_1>;
```

Name for pinctrl_lookup_state()

How To Add A New Sensor

- Step6

在cust_{project}_camera.dtsi文件中pio节点中增加如下子节点的定义，如下图：

```
camera_pins_cam0_vcamd_0: cam0@vcam0 {
    pins_cmd_dat {
        pinmux = <PINMUX_GPIO138__FUNC_GPIO138>;
        slew-rate = <1>;
        output-low;
    };
};
camera_pins_cam0_vcamd_1: cam0@vcam1 {
    pins_cmd_dat {
        pinmux = <PINMUX_GPIO138__FUNC_GPIO138>;
        slew-rate = <1>;
        output-high;
    };
};
```

0: input; 1: output

GPIO output high

```
camera_pins_cam0_vcama_0: cam0@vcam2 {
    pins_cmd_dat {
        pinmux = <PINMUX_GPIO5__FUNC_GPIO5>;
        slew-rate = <1>;
        output-low;
    };
};
camera_pins_cam0_vcama_1: cam0@vcam3 {
    pins_cmd_dat {
        pinmux = <PINMUX_GPIO5__FUNC_GPIO5>;
        slew-rate = <1>;
        output-high;
    };
};
```

GPIO output low

Set GPIO5 to GPIO mode defined at /kernel-4.14/include/dt-bindings/pinctrl/mt6885-pinfunc.h

How To Add A New Sensor

- Step6

在cust_{project}_camera.dtsi文件中kd_camera_hw1节点中将采用PMIC供电方式的pin增加如下子节点

```
cam0_vcamio-supply = <&mt_pmic_vcamio_ldo_reg>;
cam1_vcamio-supply = <&mt_pmic_vcamio_ldo_reg>;
cam2_vcamio-supply = <&mt_pmic_vcamio_ldo_reg>;
cam4_vcamio-supply = <&mt_pmic_vcamio_ldo_reg>;

status = "okay";
```

确认采用哪颗PMIC的哪个LDO，可以查看其节点名称：
kernel-4.14\arch\arm64\boot\dts\mediatek\mt6359.dtsi

```
mt_pmic_vcamio_ldo_reg: ldo_vcamio {
    regulator-name = "vcamio";
    regulator-min-microvolt = <1700000>;
    regulator-max-microvolt = <1900000>;
    regulator-enable-ramp-delay = <1920>;
};
```

How To Add A New Sensor

- Step7

如果需要指定某个sensor index对应的search list, 例如 main camera只search某些sensor，请将cust_{project}_camera.dtsi文件中kd_camera_hw1节点中添加

```
cam0_enable_sensor = "imx519_mipi_raw imx586_mipi_raw"
```

其中0对应sensor index, 后面的list则是想要search的sensor.

Note：在给sensor上电或者sensor驱动里面加延时的时候，请使用mdelay函数，不要使用msleep;否则比较容易出现启动相机时间比较久的问题。

How To Add A New Sensor

- Step8

<a>According to HW layout, Using dct tools to Config I2C device

ID	Slave Device	Channel	Device Address
13	NC		
14	SUBPMIC	I2C_CHANNEL_5	0x34
15	USB_TYPE_C	I2C_CHANNEL_5	0x4E
16	SUBPMIC_PMIC	I2C_CHANNEL_5	0x1A
17	SUBPMIC_LDO	I2C_CHANNEL_5	0x64
18	SPEAKER_AMP	I2C_CHANNEL_6	0x34
19	I2C_LCD_BIAS	I2C_CHANNEL_6	0x3E
20	DP_SWITCH	I2C_CHANNEL_6	0x50
21	EXT_BUCK_VMDDR	I2C_CHANNEL_6	0x51
22	NC		
23	NC		
24	CAMERA_MAIN	I2C_CHANNEL_8	0x1A
25	CAMERA_MAIN_EEPROM	I2C_CHANNEL_8	0x50
26	CAMERA_MAIN_AF	I2C_CHANNEL_8	0x72
27	CAMERA_MAIN_TWO_AF	I2C_CHANNEL_2	0x0C
28	NC		
29	CAMERA_MAIN_THREE	I2C_CHANNEL_9	0x10
30	CAMERA_MAIN_THREE...	I2C_CHANNEL_9	0x51
31	CAMERA_MAIN_THREE...	I2C_CHANNEL_9	0x0C

dct path:\vendor\mediatek\proprietary\scripts\dct

Dws path:\kernel-4.14\drivers\misc\mediatek\dws

表示main camera device
挂载到 I2C Bus8上面

同一条I2C bus的device
Address不能冲突
并且Address为7位，
不可以超过0X7F

Note:只需要配置一个小于0x7f的地址即可，实际该地址在代码中不使用

How To Add A New Sensor

- Step8

<b> Using DCT Tool to generate cust.dtsi file,which is included in ${projects}.dtsi.

\out\target\product\$project\obj\KERNEL_OBJ\arch\arm64\boot\dts\$project\cust.dtsi

```
&i2c2 {
    #address-cells = <1>;
    #size-cells = <0>;
    clock-frequency = <400000>;
    mediatek,use-open-drain;
    camera_main@36 {
        compatible = "mediatek,camera_main";
        reg = <0x36>;
        status = "okay";
    };
    camera_main_af@72 {
        compatible = "mediatek,camera_main_af";
        reg = <0x72>;
        status = "okay";
    };
};
```

Name of match I2C driver

Device Address

How To Add A New Sensor

- Step9

/vendor/mediatek/proprietary/custom/${project}|{platform}/hal/imgsensor_src/cfg_setting_imgsensor.cpp

```
static CUSTOM_CFG gCustomCfg[] = {
    {
        .sensorIdx      = IMGSENSOR_SENSOR_IDX_MAIN,
        .mclk           = CUSTOM_CFG_MCLK_2,
        .port           = CUSTOM_CFG_CSI_PORT_2,
        .dir            = CUSTOM_CFG_DIR_REAR,
        .bitOrder       = CUSTOM_CFG_BITORDER_9_2,
        .orientation    = 90,
        .horizontalFov  = 67,
        .verticalFov    = 49
    },
    {
        .sensorIdx      = IMGSENSOR_SENSOR_IDX_SUB,
        .mclk           = CUSTOM_CFG_MCLK_4,
        .port           = CUSTOM_CFG_CSI_PORT_0,
        .dir            = CUSTOM_CFG_DIR_FRONT,
        .bitOrder       = CUSTOM_CFG_BITORDER_9_2,
        .orientation    = 270,
        .horizontalFov  = 63,
        .verticalFov    = 40,
        .secure         = CUSTOM_CFG_SECURE_M0
    },
```

1、配置MCLK源，这里的MCLK2对应dts中的mclk1
2、配置MIPI Port Number
3、配置Rear or Front

注意：kernel底层的MCLK_0/1/2
分别对应HAL层的 MCLK_1/2/3

How To Add A New Sensor

- Step9

/vendor/mediatek/proprietary/custom/${project}|{platform}/hal/imgsensor_src/cfg_setting_imgsensor.cpp

关于Mipi Port的端口可以参考当前平台的Design Notice的MIPI C/DPHY Interface来确认硬件连接，请按照MTK Refrence Design 来硬件走线，哪一组CSI是否可以拆开使用可以具体参考每个平台的design notice。

如下为举例：CSI-0可以使用DPHY中的4lane mode以及2lane mode，也可以使用CPHY mode。

CSI #	Pad name	4-lane DPHY mode			2-lane DPHY *2 mode	CPHY mode	
		4-lane	2-lane	1-lane		3 trio	2 trio *2
CSI-0	CSI0A_L0P_T0A	RDP2	N/A	N/A	RDP0	T0A	T0A
	CSI0A_L0N_T0B	RDN2	N/A	N/A	RDN0	T0B	T0B
	CSI0A_L1P_T0C	RDP0	RDP0	RDP0	RCP	T0C	T0C
	CSI0A_L1N_T1A	RDN0	RDN0	RDN0	RCN	T1A	T1A
	CSI0A_L2P_T1B	RCP	RCP	RCP	RDP1	T1B	T1B
	CSI0A_L2N_T1C	RCN	RCN	RCN	RDN1	T1C	T1C
	CSI0B_L0P_T0A	RDP1	RDP1	N/A	RDP0	T2A	T0A
	CSI0B_L0N_T0B	RDN1	RDN1	N/A	RDN0	T2B	T0B
	CSI0B_L1P_T0C	RDP3	N/A	N/A	RCP	T2C	T0C
	CSI0B_L1N_T1A	RDN3	N/A	N/A	RCN	N/A	T1A
	CSI0B_L2P_T1B	N/A	N/A	N/A	RDP1	N/A	T1B
	CSI0B_L2N_T1C	N/A	N/A	N/A	RDN1	N/A	T1C

How To Add A New Sensor

- Step9

```
typedef enum {
  CUSTOM_CFG_CSI_PORT_0 = 0x0,     // 4D1C
  CUSTOM_CFG_CSI_PORT_1,           // 4D1C
  CUSTOM_CFG_CSI_PORT_2,           // 4D1C
  CUSTOM_CFG_CSI_PORT_3,           // 4D1C
  CUSTOM_CFG_CSI_PORT_0A,          // 2D1C or 2Trio
  CUSTOM_CFG_CSI_PORT_0B,          // 2D1C or 2Trio
  CUSTOM_CFG_CSI_PORT_1A,          // 2D1C or 2Trio
  CUSTOM_CFG_CSI_PORT_1B,          // 2D1C or 2Trio
  CUSTOM_CFG_CSI_PORT_2A,
  CUSTOM_CFG_CSI_PORT_2B,
  CUSTOM_CFG_CSI_PORT_3A,
  CUSTOM_CFG_CSI_PORT_3B,
  CUSTOM_CFG_CSI_PORT_MAX_NUM,
  CUSTOM_CFG_CSI_PORT_NONE    //for non-MIPI sensor
} CUSTOM_CFG_CSI_PORT;
```

CSI-0
CSI-1
CSI-2
CSI-3

CSI-0A
CSI-0B
…
…
某个CSI是否支持拆分需要参考Design Notice

How To Add A New Sensor

- Step10

根据各个接口函数，在sample code基础上进行修改。

- Kernel:

  /kernel-4.14/drivers/misc/mediatek/imgsensor/src/common/v1_1/xxx_mipi_raw/

- Hal:

  a) Tuning parameter:

  /vendor/mediatek/proprietary/custom/${platform}/hal/imgsensor/ver1/xxx_mipi_raw/

  b) Metadata:

  /vendor/mediatek/proprietary/custom/common/hal/imgsensor_metadata/sensor/xxx_mipi_raw/

  /vendor/mediatek/proprietary/custom/ /${platform}/hal/imgsensor_metadata/xxx_mipi_raw/

Note：

1、Tuning Parameter与Metadata可以采用附录作为参考，参考FAQ22441复制对应文件夹并将其中所有的Sensor Name、Sensor Id等更改成当前需要porting的sensor的对应名称。

2、Kernel 版本号可以根据当前Project的ProjectConfig.mk中的LINUX_KERNEL_VERSION得知。Driver是v1_1或者其他版本可以通过Platform下的Makefile中的COMMON_VERSION得知。

How To Add A New Sensor

- Step11

  - Modify projectConfig.mk，重新full build
    Eg: make clean && make –j16 2>&1 | tee build.log

  - Modify kernel part, remake kernel bootimage, 只下载boot.img即可
    Eg: mmm kernel-4.14:kernel && make bootimage 2>&1 | tee kernel.log

  - Modify hal part, remake android, 下载vendor.img 或者push对应的so即可。
    Eg: make vendorimage 2>&1 | tee vendor.log

Note:
1、编译的时候注意文件的优先级 project>platform
2、若点不亮Sensor，请按照FAQ22328 Check

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity、OB)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

Driver Data Structure Introduction

xxxmipiraw_Sensor.h：

1 记录**sensor info常量**的结构体 → **imgsensor_info_struct**

2 记录**sensor info变量**的结构体 → **imgsensor_struct**

3 记录sensor mode info的结构体 → **imgsensor_mode_struct**

```
/* SENSOR PRIVATE STRUCT FOR CONSTANT*/
struct imgsensor_info_struct {
        kal_uint32 sensor_id; /* record sensor id defined in Kd_imgsensor.h */
        kal_uint32 checksum_value; /* checksum value for Camera Auto Test */
        struct imgsensor_mode_struct pre;
        struct imgsensor_mode_struct cap;
        struct imgsensor_mode_struct normal_video;
        struct imgsensor_mode_struct hs_video;
        struct imgsensor_mode_struct slim_video;
        struct imgsensor_mode_struct custom1;
        struct imgsensor_mode_struct custom2;
        struct imgsensor_mode_struct custom3;

        kal_uint8 ae_shut_delay_frame; /* shutter delay frame for AE cycle *
        kal_uint8 ae_sensor_gain_delay_frame;
        kal_uint8 ae_ispGain_delay_frame;
        kal_uint8 ihdr_support; /* 1, support; 0,not support */
        kal_uint8 ihdr_le_firstline; /* 1,le first ; 0, se first */
        kal_uint8 temperature_support;
        kal_uint8 sensor_mode_num; /* support sensor mode num */

        kal_uint8 cap_delay_frame; /* enter capture delay frame num */
        kal_uint8 pre_delay_frame; /* enter preview delay frame num */
        kal_uint8 video_delay_frame; /* enter video delay frame num */
        kal_uint8 hs_video_delay_frame;
        kal_uint8 slim_video_delay_frame; /* enter slim video delay frame nu
        kal_uint8 custom1_delay_frame; /* enter custom1 delay frame num */
        kal_uint8 custom2_delay_frame; /* enter custom2 delay frame num */
        kal_uint8 custom3_delay_frame; /* enter custom3 delay frame num */
        kal_uint8  frame_time_delay_frame;
        kal_uint8 margin; /* sensor framelength & shutter margin */
        kal_uint32 min_shutter; /* min shutter */
        kal_uint32 min_gain;
        kal_uint32 max_gain;
        kal_uint32 min_gain_iso;
        kal_uint32 gain_step;
        kal_uint32 gain_type;
        kal_uint32 max_frame_length;
        kal_uint8 isp_driving_current; /* mclk driving current */
        kal_uint8 sensor_interface_type; /* sensor_interface_type *
        kal_uint8 mipi_sensor_type;
        /* 0,MIPI_OPHY_NCSI2; 1,MIPI_OPHY_CSI2, default is NCSI2,
         * don't modify this para
         */
        kal_uint8 mipi_settle_delay_mode;
        /* 0, high speed signal auto detect;
         * 1, use settle delay,unit is ns,
         * default is auto detect, don't modify this para
         */
        kal_uint8 sensor_output_dataformat;
        kal_uint8 mclk;  /* mclk value, suggest 24 or 26 for 24Mhz
        kal_uint32 i2c_speed; /* i2c speed */
        kal_uint8 mipi_lane_num; /* mipi lane num */
        kal_uint8 i2c_addr_table[5];
```

Driver Data Structure Introduction

xxxmipiraw_Sensor.h：

1 记录**sensor info常量**的结构体 → **imgsensor_info_struct**

2 记录**sensor info变量**的结构体 → **imgsensor_struct**

3 记录sensor mode info的结构体 → **imgsensor_mode_struct**

```
struct imgsensor_struct {
        kal_uint8 mirror; /* mirrorflip information */

        kal_uint8 sensor_mode; /* record IMGSENSOR_MODE enum value */

        kal_uint32 shutter; /* current shutter */
        kal_uint16 gain; /* current gain */

        kal_uint32 pclk; /* current pclk */

        kal_uint32 frame_length; /* current framelength
        kal_uint32 line_length; /* current linelength *

        kal_uint32 min_frame_length;
        kal_uint16 dummy_pixel; /* current dummypixel *
        kal_uint32 dummy_line; /* current dummline */

        kal_uint16 current_fps; /* current max fps */
        kal_bool autoflicker_en; /* record autoflicker
        kal_bool test_pattern; /* record test pattern m
        enum MSDK_SCENARIO_ID_ENUM current_scenario_id;
        kal_bool ihdr_en; /* ihdr enable or disable */
        kal_uint8 ihdr_mode; /* ihdr enable or disable
        kal_uint8 pdaf_mode; /* ihdr enable or disable */
        kal_uint8 i2c_write_id; /* record current sensor's i2c write id */
        struct IMGSENSOR_AE_FRM_MODE ae_frm_mode;
        kal_uint8 current_ae_effective_frame;
};
```

```
struct imgsensor_mode_struct {
        kal_uint32 pclk; /* record different mode's pclk */
        kal_uint32 linelength; /* record different mode's linelength */
        kal_uint32 framelength; /* record different mode's framelength */

        kal_uint8 startx; /* record different mode's startx of grabwindow */
        kal_uint8 starty; /* record different mode's startx of grabwindow */

        kal_uint16 grabwindow_width;
        kal_uint16 grabwindow_height;

/* for MIPIDataLowPwr2HighSpeedSettleDelayCount by different scenario*/
        kal_uint8 mipi_data_lp2hs_settle_dc;

/*     following for GetDefaultFramerateByScenario()    */
        kal_uint16 max_framerate;
        kal_uint32 mipi_pixel_rate;
};
```

Driver Data Structure Introduction

imgsensor_info_struct的成员解析

Member	Meaning
sensor_id	Record sensor id define in kd_imgsensor.h
checksum_value	Test Pattern输出时计算出来的checksum值,用于 Camera Auto Test
Imgsensor_mode_struct	pre,cap,normal_video,hs_video,slim_video, custom1, custtom2……
ae_shutter_delay_frame/ae_sensor_gain_delay_frame/ae_isp_gain_delay_frame	shutter, sensor gain, isp gain想要在同一帧生效的话，分别需要在第几帧设置，从cnt 0开始计算。 Ex: shutter，sensor gain都是当前帧设下去，两帧后生效；而isp各个平台上都是当前帧生效，故三个值依次分别设置为0,0,2
ihdr_support	sensor interleave mode是否支援？1: support, 0: not support
ihdr_le_firstline	如果支持ihdr，第一行是长曝le还是短曝se？1: le first, 0: se first
temperature_support	是否支持get sensor的温度
sensor_mode_num	支持的sensor mode个数，切记与当前winsizeinfo、setting的个数保持一致
cap_delay_frame/pre_delay_frame…	丢掉不稳定的帧，避免因sensor切mode后还没有稳定而看到异常画面
frame_time_delay_frame	frame length 几帧生效,Sony Sensor请填3，其它Sensor请填2
Margin	来自Sensor Datasheet

Driver Data Structure Introduction

imgsensor_info_struct的成员解析

Member	Meaning
min_shutter	shutter可以写的最小值 (需要注意sensor 对shutter是否有奇偶要求)
min_gain/max_gain	Sensor所支持的最小gain与最大gain，64代表1倍gain
main_gain_iso	ISO value at 1x gain
gain_step	Minimum valid gain step
gain_type	Sony:type 0; OV:type 1; Samsung:type 2; Hinyx:type 3; GC:type 4
max_frame_length	framelength可以写的最大值，要考虑到framelength寄存器的位数，max shutter会根据max_framelength-margin做保护
isp_driving_current	用来调整mclk的驱动电流(2,4,6,8MA)
sensor_interface_type	0:SENSOR_INTERFACE_TYPE_PARALLEL; 1:SENSOR_INTERFACE_TYPE_MIPI
mipi_sensor_type	0:MIPI_OPHY_NCSI2 ; 2:MIPI_CPHY
sensor_output_dataformat	参考kd_imgsensor_define.h RAW/RAW8/YUV/RAW_4CELL……
mclk	建议24Mhz or 26Mhz, 24 means 24Mhz, 26 means 26Mhz，一般使用24Mhz. (Vendor提供setting的时候要注意请vendor以24Mhz的mclk提供setting)
i2c_speed	默认400/1000 根据需要调整(需要确认driver是否有用到)
mipi_lane_num	MIPI sensor使用的lane num （HW请优先使用低位的lane接口）
i2c_addr_table	i2c write id，以0xff结尾，最多四组。Ex.i2c_addr_table = {0x45, 0x44,0xff},

Driver Data Structure Introduction

imgsensor_mode_struct的成员解析

Member	Meaning
pclk	结合 linelength, framelength用来计算linetime和framerate.
linelength	valid pixels+dummy pixels
framelength	valid lines+dummy lines
Startx/starty	BB TG grab sensor output window's offset, 建议设为0,0， 除非调视角问题需要改动这里
grabwindow_width grabwindow_height	BB TG grab window size,建议设为sensor实际输出的宽高
mipi_data_lp2hs_settle_dc	目前已弃用，保持默认值不需要修改
max_framerate	10base, eg: this mode support max 30fps, you should config as 300
mipi_pixel_rate	请按照FAQ22328/FAQ21124使用Sentest测试每颗sensor的每个mode，将该值的测试结果填写到sensor driver中

Driver Data Structure Introduction

- imgsensor_info_struct成员初始化

```
static struct imgsensor_info_struct imgsensor_info = {
        .sensor_id = IMX519_SENSOR_ID,

        .checksum_value = 0x8ac2d94a,
        .pre = {
                .pclk = 500000000,
                .linelength = 5920,
                .framelength = 2815,
                .startx = 0,
                .starty = 0,
                .grabwindow_width = 2328,
                .grabwindow_height = 1728, /*1746*/
                .mipi_data_lp2hs_settle_dc = 85,
                /* following for GetDefaultFramerateB
                .mipi_pixel_rate = 226290000,
                .max_framerate = 300, /* 30fps */
        },


.min_gain = 64, /*1x gain*/
.max_gain = 1024, /*16x gain*/
.min_gain_iso = 100,
.margin = 32,           /* sensor framelength & shut
.min_shutter = 2,       /* min shutter */
.gain_step = 1,
.gain_type = 0,
.max_frame_length = 0xffff,
.ae_shut_delay_frame = 0,
.ae_sensor_gain_delay_frame = 0,
.ae_ispGain_delay_frame = 2,    /* isp gain delay fr
.ihdr_support = 0,      /* 1, support; 0,not support
.ihdr_le_firstline = 0, /* 1,le first ; 0, se first
.temperature_support = 1, /* 1, support; 0,not support
.sensor_mode_num = 8,   /* support sensor mode num */

        .cap_delay_frame = 2,   /* enter capture delay frame num */
        .pre_delay_frame = 2,   /* enter preview delay frame num */
        .video_delay_frame = 2, /* enter video delay frame num */
        .hs_video_delay_frame = 2,
        .slim_video_delay_frame = 2,    /* enter slim video delay frame num */
        .custom1_delay_frame = 2,       /* enter custom1 delay frame num */
        .custom2_delay_frame = 2,       /* enter custom2 delay frame num */
        .custom3_delay_frame = 2,

        .isp_driving_current = ISP_DRIVING_4MA,
        .sensor_interface_type = SENSOR_INTERFACE_TYPE_MIPI,
        /* .mipi_sensor_type = MIPI_OPHY_NCSI2, */
        /* 0,MIPI_OPHY_NCSI2;  1,MIPI_OPHY_CSI2 */
        .mipi_sensor_type = MIPI_CPHY, /* 0,MIPI_OPHY_NCSI2; 1,MIPI_OPHY_CSI2 */
        .mipi_settle_delay_mode = 0,
        .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_RAW_B,
#if IMX519_CAP_2TRIO
        .mclk = 6, /* 6MHz mclk for 26fps */
        .mipi_lane_num = SENSOR_MIPI_2_LANE,
#else
        .mclk = 24, /* mclk value, suggest 24 or 26 for 24Mhz or 26Mhz */
        /*.mipi_lane_num = SENSOR_MIPI_4_LANE,*/
        .mipi_lane_num = SENSOR_MIPI_3_LANE,
#endif
        .i2c_addr_table = {0x34, 0xff},
        /* record sensor support all write id addr,
         * only supprt 4 must end with 0xff
         */
        .i2c_speed = 1000, /* i2c read/write speed */
};
```

Driver Data Structure Introduction

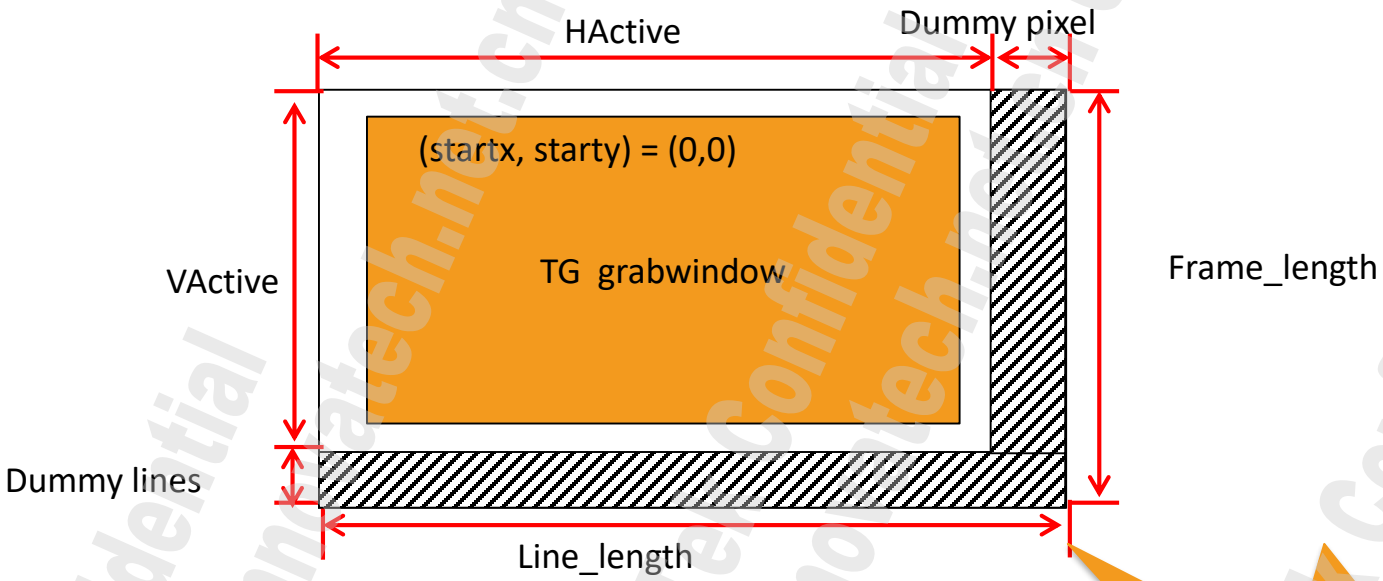
关于i2c speed的修改

```
static void write_cmos_sensor(kal_uint16 addr, kal_uint16 para)
{
        char pusendcmd[4] = {(char)(addr >> 8), (char)(addr & 0xFF),
                             (char)(para >> 8), (char)(para & 0xFF)};
        iWriteRegI2C(pusendcmd, 4, imgsensor.i2c_write_id);
}
```

默认使用的的API(iWriteRegI2C)中没有使用到驱动中填的i2c_speed,默认走400Kbps的速率，若有需要使用到1M的速率的需求，可将imgsensor_info.i2c_speed设定为1000并更换为如下API：

```
iWriteRegI2CTiming(pusendcmd, 4, imgsensor.i2c_write_id,imgsensor_info.i2c_speed);
```

Driver Data Structure Introduction



Vblanking=(frame_length- grabwindow_height)*line_length/PCLK

ISP6s之前的平台要求Vblanking > 1ms

ISP6s平台要求Vertical blanking Spec

- fps <= 30 ➡ VB > 1ms
- 30 < fps < 120 ➡ VB > 650us
- fps >= 120 ➡ VB > 350us

Must Have

VB: The interval between the last line of data and the start of next frame

Driver Data Structure Introduction

imgsensor_struct的成员解析

Member	Meaning
mirror	记录当前是normal, or H mirror, or V flip,or both H& V.
sensor_mode	记录当前处于哪种mode(init/preview/capture/video)
shutter	记录当前的shutter值，unit: line
gain	记录当前的sensor gain值，unit: 64代表1X
pclk	记录当前mode的pclk值，unit: hz, eg:168000000表示168Mhz
frame_length/line_length	记录当前的framelength, linelength(同种mode 下linelength一般不会动态变化)
min_frame_length	记录当前mode下最大帧率对应的最小framelength.当shutter小于min_frame_length时，为确保最大帧率不变，framelength不能小于min_frame_length.
dummy_pixel/dummy_line	记录当前的dummy pixel, dummy line的值

Driver Data Structure Introduction

imgsensor_struct的成员解析

Member	Meaning
current_fps	记录当前mode的最大帧率。Unit: 10base， eg: 240表示24fps
autoflicker_en	记录当前disable还是enable auto flicker, 该值上层会带下来
test_pattern	记录当前是否让sensor输出test pattern， factory mode下camera auto test时会调用test pattern函数
current_scenario_id	记录当前处于哪个scenario（preview/capture/video）
ihdr_en	记录是否要enable or disable ihdr. 上层带下来的参数，如果sensor支持interleave mode，当点击hdr icon时，会带该参数下来传给driver，用来判断是否要senosr跑ihdr setting
ihdr_mode	0:None 1:RAW 2:CAMSV
pdaf_mode	记录sensor当前的pdaf mode
i2c_write_id	记录sensor当前i2c通信使用的write id
ae_frm_mode	Used by long exposure -> FAQ21536
current_ae_effective_frame	用于长曝光，记录sensor长曝光是N+1还是N+2生效

Driver Data Structure Introduction

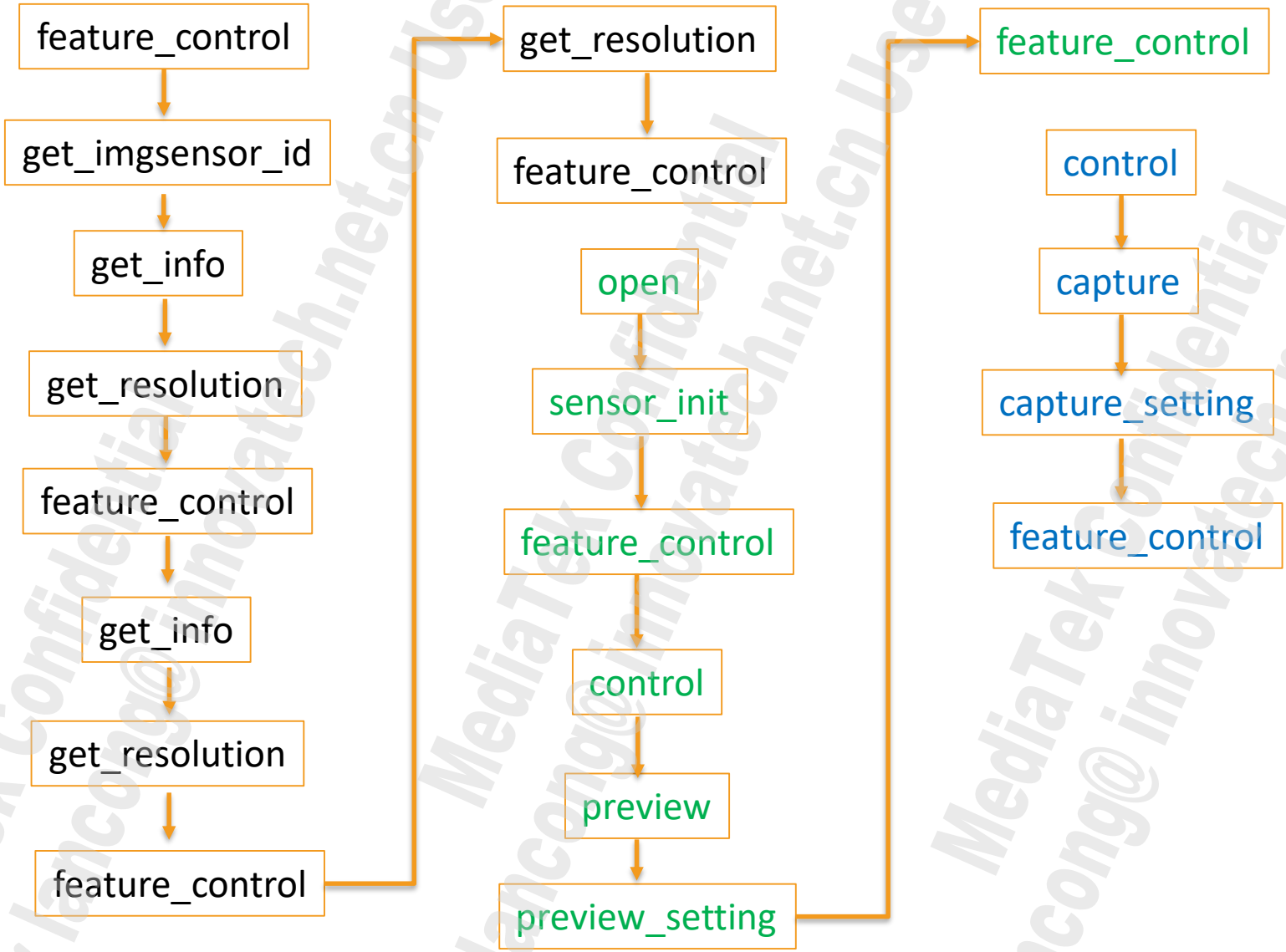
- imgsensor_struct初始化

```
static struct imgsensor_struct imgsensor = {
    .mirror = IMAGE_NORMAL, /* mirrorflip information */
    .sensor_mode = IMGSENSOR_MODE_INIT,
    /* IMGSENSOR_MODE enum value,record current sensor mode,such as:
     * INIT, Preview, Capture, Video,High Speed Video, Slim Video
     */
    .shutter = 0x3D0,    /* current shutter */
    .gain = 0x100,       /* current gain */
    .dummy_pixel = 0,    /* current dummypixel */
    .dummy_line = 0,     /* current dummyline */
    .current_fps = 300,
    .autoflicker_en = KAL_FALSE,
    .test_pattern = KAL_FALSE,
    .current_scenario_id = MSDK_SCENARIO_ID_CAMERA_PREVIEW,
    .ihdr_mode = 0, /* sensor need support LE, SE with HDR feature */
    .i2c_write_id = 0x34, /* record current sensor's i2c write id */
};
```

Sensor Mode Introduction

Mode	Introduction
Preview	Preview size(<3M, use full size 3M; >3M, use ¼ full size binning average is prefered)@30fps
Capture	Full size@30fps, For capture , ZSD preview
Normal Video	Full size@30fps, 如果full size达不到30fps, 就用30fps可以达到的最大size的setting, 建议至少4K2K(3840*2160)，这样就可以支持4K2K encode. 若开启lowPowerVideoRecord，小于4K2K的video size就会走preview setting.
HS Video	快录慢播（slow motion）请参照platform spec FHD@120fps>HD@180fps>HD@120fps>VGA@120fps
Slim Video	for VT, 视频通话，低功耗，720P@30fps
Custom1	按需配置，一般用于双摄
Custom2	按需配置
Custom3	按需配置

Search→open→preview→capture 函数出现顺序



Driver Functions

1、Driver函数入口

- XXX_MIPI_RAW_SensorInit.
- Driver函数的入口，用来获取sensor_func函数指针，用来后续query sensor info和control等的控制.

```
UINT32 IMX519_MIPI_RAW_SensorInit(struct SENSOR_FUNCTION_STRUCT **pfFunc)
{
        /* To Do : Check Sensor status here */
        if (pfFunc != NULL)
                *pfFunc = &sensor_func;
        return ERROR_NONE;
} /* IMX519_MIPI_RAW_SensorInit */



static struct SENSOR_FUNCTION_STRUCT sensor_func = {
        open,
        get_info,
        get_resolution,
        feature_control,
        control,
        close
};
```

Driver Functions

确定I2C是否ok??

2、get_imgsensor_id函数

开机search sensor时会通过get_imgsensor_id函数读取id. 若能成功读到id，在UI上显示camera app的Icon.

```
static kal_uint32 get_imgsensor_id(UINT32 *sensor_id)
{
        kal_uint8 i = 0;
        kal_uint8 retry = 2;
        /*sensor have two i2c address 0x6c 0x6d & 0x21 0x20,
         *we should detect the module used i2c address
         */
        while (imgsensor_info.i2c_addr_table[i] != 0xff) {
                spin_lock(&imgsensor_drv_lock);
                imgsensor.i2c_write_id = imgsensor_info.i2c_addr_table[i];
                spin_unlock(&imgsensor_drv_lock);
                do {
                        *sensor_id = ((read_cmos_sensor_8(0x0016) << 8)
                                        | read_cmos_sensor_8(0x0017));
                        if (*sensor_id == imgsensor_info.sensor_id) {
                                LOG_INF("i2c write id: 0x%x, sensor id: 0x%x\n",
                                        imgsensor.i2c_write_id, *sensor_id);
                                return ERROR_NONE;
                        }

                        LOG_INF("Read sensor id fail, id: 0x%x\n",
                                imgsensor.i2c_write_id);
                        retry--;
                } while (retry > 0);
                i++;
                retry = 2;
        }
        if (*sensor_id != imgsensor_info.sensor_id) {
                /*if Sensor ID is not correct,
                 *Must set *sensor_id to 0xFFFFFFFF
                 */
                *sensor_id = 0xFFFFFFFF;
                return ERROR_SENSOR_CONNECT_FAIL;
        }
        return ERROR_NONE;
}
```

从sensor读取sensor id

影响开机速度：
1. I2c write id 的兼容，
2. Retry次数

如何做到 Camera app icon 始终显示在UI上

```
static kal_uint32 get_imgsensor_id(UINT32 *sensor_id)
{
        *sensor_id = 0x3000;
         return ERROR_NONE;
}
```

参照datasheet修改

3、 open函数

```
static kal_uint32 open(void)
{
        kal_uint8 i = 0;
        kal_uint8 retry = 2;
        kal_uint16 sensor_id = 0;

        /*sensor have two i2c address 0x6c 0x6d & 0x21 0x20,
         *we should detect the module used i2c address
         */
        while (imgsensor_info.i2c_addr_table[i] != 0xff) {
                spin_lock(&imgsensor_drv_lock);
                imgsensor.i2c_write_id = imgsensor_info.i2c_addr_table[i];
                spin_unlock(&imgsensor_drv_lock);
                do {
                        sensor_id = ((read_cmos_sensor_8(0x0016) << 8)
                                        | read_cmos_sensor_8(0x0017));
                        if (sensor_id == imgsensor_info.sensor_id) {
                                LOG_INF("i2c write id: 0x%x, sensor id: 0x%x\n",
                                        imgsensor.i2c_write_id, sensor_id);
                                break;
                        }
                        LOG_INF("Read sensor id fail, id: 0x%x\n",
                                imgsensor.i2c_write_id);
                        retry--;
                } while (retry > 0);
                i++;
                if (sensor_id == imgsensor_info.sensor_id)
                        break;
                retry = 2;
        }
        if (imgsensor_info.sensor_id != sensor_id)
                return ERROR_SENSOR_CONNECT_FAIL;

        /* initail sequence write in  */
        sensor_init();

        spin_lock(&imgsensor_drv_lock);

        imgsensor.autoflicker_en = KAL_FALSE;
        imgsensor.sensor_mode = IMGSENSOR_MODE_INIT;
        imgsensor.shutter = 0x3D0;
        imgsensor.gain = 0x100;
        imgsensor.pclk = imgsensor_info.pre.pclk;
        imgsensor.frame_length = imgsensor_info.pre.framelength;
        imgsensor.line_length = imgsensor_info.pre.linelength;
        imgsensor.min_frame_length = imgsensor_info.pre.framelength;
        imgsensor.dummy_pixel = 0;
        imgsensor.dummy_line = 0;
        imgsensor.ihdr_mode = 0;
        imgsensor.test_pattern = KAL_FALSE;
        imgsensor.current_fps = imgsensor_info.pre.max_framerate;
        spin_unlock(&imgsensor_drv_lock);

        return ERROR_NONE;
} /* open */
```

1)每次进camera时会调用
2)读sensorid, 确认i2c通信是否正常.
3)调用sensor_init函数,
4)初始化Imgsensor结构体中的一些变量
5)Otp 读取

Driver Functions

4、 sensor_init 函数

```
static void sensor_init(void)
{
        LOG_INF("E\n");
        imx519_table_write_cmos_sensor(imx519_init_setting,
                sizeof(imx519_init_setting)/sizeof(kal_uint16));
```

初始化设定(PLL, MIPI config,size,…)

```
write_cmos_sensor_8(0x0100, 0x00);
```

init后不要让sensor输出数据，要进入standby状态,

1. Sensor datasheet 会有类似要求
2. 避免BB端sensor interface, isp启动未完成，造成数据接收异常.

Driver Functions

5、close函数

```
static kal_uint32 close(void)
{
    pr_debug("E\n");
    /* No Need to implement this function */
    streaming_control(KAL_FALSE);
    return ERROR_NONE;
} /* close */
```

Driver Functions

6、 get_info （不需要修改）

```
static kal_uint32 get_info(enum MSDK_SCENARIO_ID_ENUM scenario_id,
                           MSDK_SENSOR_INFO_STRUCT *sensor_info,
                           MSDK_SENSOR_CONFIG_STRUCT *sensor_config_data)
{

        sensor_info->SensorClockPolarity = SENSOR_CLOCK_POLARITY_LOW;
        sensor_info->SensorClockFallingPolarity = SENSOR_CLOCK_POLARITY_LOW;
        sensor_info->SensorHsyncPolarity = SENSOR_CLOCK_POLARITY_LOW;
        sensor_info->SensorVsyncPolarity = SENSOR_CLOCK_POLARITY_LOW;
        sensor_info->SensorInterruptDelayLines = 4; /* not use */
        sensor_info->SensorResetActiveHigh = FALSE; /* not use */
        sensor_info->SensorResetDelayCount = 5; /* not use */

        sensor_info->SensroInterfaceType = imgsensor_info.sensor_interface_typ
        sensor_info->MIPIsensorType = imgsensor_info.mipi_sensor_type;
        sensor_info->SettleDelayMode = imgsensor_info.mipi_settle_delay_mode;
        sensor_info->SensorOutputDataFormat =
                imgsensor_info.sensor_output_dataformat;

        sensor_info->CaptureDelayFrame = imgsensor_info.cap_delay_frame;
        sensor_info->PreviewDelayFrame = imgsensor_info.pre_delay_frame;
        sensor_info->VideoDelayFrame = imgsensor_info.video_delay_frame;
        sensor_info->HighSpeedVideoDelayFrame =
                imgsensor_info.hs_video_delay_frame;
        sensor_info->SlimVideoDelayFrame =
                imgsensor_info.slim_video_delay_frame;
        sensor_info->Custom1DelayFrame = imgsensor_info.custom1_delay_frame;
        sensor_info->Custom2DelayFrame = imgsensor_info.custom2_delay_frame;
        sensor_info->Custom3DelayFrame = imgsensor_info.custom3_delay_frame;

        sensor_info->SensorMasterClockSwitch = 0; /* not use */
        sensor_info->SensorDrivingCurrent = imgsensor_info.isp_driving_current
        sensor_info->AEShutDelayFrame = imgsensor_info.ae_shut_delay_frame;
        sensor_info->AESensorGainDelayFrame =
                imgsensor_info.ae_sensor_gain_delay_frame;
        sensor_info->AEISPGainDelayFrame =
                imgsensor_info.ae_ispGain_delay_frame;
        sensor_info->IHDR_Support = imgsensor_info.ihdr_support;
        sensor_info->IHDR_LE_FirstLine = imgsensor_info.ihdr_le_firstline;
        sensor_info->TEMPERATURE_SUPPORT = imgsensor_info.temperature_support;
        sensor_info->SensorModeNum = imgsensor_info.sensor_mode_num;
        sensor_info->PDAF_Support = 2;
        sensor_info->SensorMIPILaneNumber = imgsensor_info.mipi_lane_num;
        sensor_info->SensorClockFreq = imgsensor_info.mclk;
        sensor_info->SensorClockDividCount = 3; /* not use */
        sensor_info->SensorClockRisingCount = 0;
        sensor_info->SensorClockFallingCount = 2; /* not use */
        sensor_info->SensorPixelClockCount = 3; /* not use */
        sensor_info->SensorDataLatchCount = 2; /* not use */

        sensor_info->MIPIDataLowPwr2HighSpeedTermDelayCount = 0;
        sensor_info->MIPICLKLowPwr2HighSpeedTermDelayCount = 0;
        sensor_info->SensorWidthSampling = 0; /* 0 is default 1x */
        sensor_info->SensorHightSampling = 0; /* 0 is default 1x */
        sensor_info->SensorPacketECCOrder = 1;

        switch (scenario_id) {
        case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
                sensor_info->SensorGrabStartX = imgsensor_info.pre.startx;
                sensor_info->SensorGrabStartY = imgsensor_info.pre.starty;

                sensor_info->MIPIDataLowPwr2HighSpeedSettleDelayCount =
                        imgsensor_info.pre.mipi_data_lp2hs_settle_dc;

                break;
        case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
                sensor_info->SensorGrabStartX = imgsensor_info.cap.startx;
                sensor_info->SensorGrabStartY = imgsensor_info.cap.starty;

                sensor_info->MIPIDataLowPwr2HighSpeedSettleDelayCount =
                        imgsensor_info.cap.mipi_data_lp2hs_settle_dc;

                break;
        ......
```

Driver Functions

7、feature_control

7.1

SENSOR_FEATURE_GET_GAIN_RANGE_BY_SCENARIO	New Must
SENSOR_FEATURE_GET_BASE_GAIN_ISO_AND_STEP	New Must
SENSOR_FEATURE_GET_MIN_SHUTTER_BY_SCENARIO	New Must
SENSOR_FEATURE_GET_BINNING_TYPE	New Must
SENSOR_FEATURE_GET_FRAME_CTRL_INFO_BY_SCENARIO	New Must
SENSOR_FEATURE_GET_ANA_GAIN_TABLE	New Can be added if needed
SENSOR_FEATURE_GET_AWB_REQ_BY_SCENARIO	New Can be added if needed

```
case SENSOR_FEATURE_GET_GAIN_RANGE_BY_SCENARIO:
    *(feature_data + 1) = imgsensor_info.min_gain;
    *(feature_data + 2) = imgsensor_info.max_gain;
    break;
case SENSOR_FEATURE_GET_BASE_GAIN_ISO_AND_STEP:
    *(feature_data + 0) = imgsensor_info.min_gain_iso;
    *(feature_data + 1) = imgsensor_info.gain_step;
    *(feature_data + 2) = imgsensor_info.gain_type;
    break;
case SENSOR_FEATURE_GET_MIN_SHUTTER_BY_SCENARIO:
    *(feature_data + 1) = imgsensor_info.min_shutter;
    *(feature_data + 2) = imgsensor_info.exp_step;
    break;
```

Gain type:

1、Sony, type 0
   need a gain table
   due to the gain table are the same among sensors
   type 0 will not need to pass gain table
2、OV, type 1
   write (gain value)*2 to gain register of sensor
   gain step was 1
3、Samsung, type2
   write (gain value)/2 to gain register of sensor
   gain step was 2
4、Hynix, type 3
   gain step was 4, sensor list: hynix846
5、GC, type4
   gain step was 1, sensor list: gc02m0

*What we are talking about here are analog gain, gain step was for kermel space settings

Note：ISP6s以及之后的平台需要实作，用来Auto Generate Deviceinfo

New command for AE mgr to gen device info

- SENSOR_FEATURE_GET_GAIN_RANGE_BY_SCENARIO
  - arg0 = scenario id, arg1 = min gain, arg2 = max gain
  - 64→1x gain, 1024→16x gain and so on…
- SENSOR_FEATURE_GET_BASE_GAIN_ISO_AND_STEP
  - arg0 = min gain iso, arg1 = gain step, arg2 = gain type
  - gain step = 1 or 2 or 3 or 4…
- SENSOR_FEATURE_GET_MIN_SHUTTER_BY_SCENARIO
  - arg0 = scenario id, arg1 = min shutter
  - Min shutter→ in line time unit
- SENSOR_FEATURE_GET_BINNING_TYPE
  - Return binning ratio by sensor mode.
  - The case should be implemented when there is sensor mode with binning summed.
- SENSOR_FEATURE_GET_FRAME_CTRL_INFO_BY_SCENARIO
  - Please assign 1 to first parameter(this is used to indicate feature is supported or not).
  - and margin value to second parameter(that margin value could be determined by scenario)
- SENSOR_FEATURE_GET_ANA_GAIN_TABLE
  - arg0 = size of table (byte), arg1 = pointer to buffer
  - 1st time to get size, 2nd to get data (user need to allocate buffer)
- SENSOR_FEATURE_GET_AWB_REQ_BY_SCENARIO
  - The user can obtain whether a sensor mode needs to write AWB gain according to this parameter

```
.min_gain = 64, /*1x gain*/
.max_gain = 1024, /*16x gain*/
.min_gain_iso = 100,
.margin = 32,           /* sensor framelength & shutter margin */
.min_shutter = 2,       /* min shutter */
.gain_step = 1,
.gain_type = 0,
```

New command for AE mgr to gen device info

How to dump deviceinfo

Before enter camera APK

adb root
adb shell
mkdir /data/vendor/seninfo
adb shell setprop vendor.sensor.device.dump.enable 1
Enter the camera from camera APK that you want to dump*1
dump the device info
adb pull /data/vendor/seninfo [PC path]

```
case SENSOR_FEATURE_GET_BINNING_TYPE:
    switch (*(feature_data + 1)) {
    case MSDK_SCENARIO_ID_CUSTOM3:
        *feature_return_para_32 = 1; /*BINNING_NONE*/
        break;
    case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
    case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
    case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
    case MSDK_SCENARIO_ID_SLIM_VIDEO:
    case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
    case MSDK_SCENARIO_ID_CUSTOM4:
    default:
        *feature_return_para_32 = 2; /*BINNING_AVERAGED*/
        break;
    }
    pr_debug("SENSOR_FEATURE_GET_BINNING_TYPE AE_binning_type:%d,\n",
        *feature_return_para_32);
    *feature_para_len = 4;

    break;
```

需要sensor vendor告知每个Sensor Mode的binnig ratio来填到对应的mode中。

Eg：custom3为full size，其他mode为binning size。由于存在binning ratio，相同的设定下，binning size下的亮度是full size的两倍，所以fullsize的custom3 mode返回值设定为1，其他mode返回值设定为2。

若不同的mode存在不同的binning ratio，在驱动中设定之后也需要确认tuning参数中camera_AE_custom_transform.cpp中的transBinSum_{sensor name}mipiraw这个API是否实现，若没有实现，请参考其他颗sensor的实现方式来实现，eg:OV48B

```
case SENSOR_FEATURE_GET_FRAME_CTRL_INFO_BY_SCENARIO:
    /*
     * 1, if driver support new sw frame sync
     * set_shutter_frame_length() support third para auto_extend_en
     */
    *(feature_data + 1) = 1;
    /* margin info by scenario */
    *(feature_data + 2) = imgsensor_info.margin;
    break;
```

```
case SENSOR_FEATURE_GET_AWB_REQ_BY_SCENARIO:
    switch (*feature_data) {
    case MSDK_SCENARIO_ID_CUSTOM3:
        *(MUINT32 *)(uintptr_t)(*(feature_data + 1)) = 1;
        break;
    default:
        *(MUINT32 *)(uintptr_t)(*(feature_data + 1)) = 0;
        break;
    }
    break;
```

```
case SENSOR_FEATURE_GET_ANA_GAIN_TABLE:
    if ((void *)(uintptr_t) (*(feature_data + 1)) == NULL
                            || *(feature_data + 0) == 0) {
        *(feature_data + 0) =
            sizeof(ana_gain_table)/sizeof(char);
    } else {
        memcpy((void *)(uintptr_t) (*(feature_data + 1)),
        (void *)ana_gain_table,
        sizeof(ana_gain_table)/sizeof(char));
    }
    break;
```

Driver Functions

7.2

SENSOR_FEATURE_SET_MCLK_DRIVE_CURRENT	Set driver sensor MCLK driving current. Ex.2mA/4mA/6mA/...
SENSOR_FEATURE_GET_VC_INFO2	*There are new structure and enum for VC_INFOR2.
SENSOR_FEATURE_GET_PIXEL_CLOCK_FREQ_BY_SCENARIO	After isp60 Must FAQ21484
SENSOR_FEATURE_GET_PERIOD_BY_SCENARIO	After isp60 Must FAQ21484

7.3、Feature id: SENSOR_FEATURE_GET_PERIOD

```
case SENSOR_FEATURE_GET_PERIOD:
    *feature_return_para_16++ = imgsensor.line_length;
    *feature_return_para_16 = imgsensor.frame_length;
    *feature_para_len = 4;
    break;
```

获取各个mode的sensor linelength, framelength

- **linelength,** 结合pclk来计算linetime, 用来换算ae plinetable的exposure time对应的shutter (unit: line,条数)

7.4 、 Feature id: SENSOR_FEATURE_GET_PIXEL_CLOCK_FREQ

```
case SENSOR_FEATURE_GET_PIXEL_CLOCK_FREQ:
    *feature_return_para_32 = imgsensor.pclk;
    *feature_para_len = 4;
    break;
```

- 获取各个mode的sensor pclk, (pixel/s)
- 与linelength用来换算ae plinetable的exposure time对应的shutter

7.5、Feature id: SENSOR_FEATURE_SET_ESHUTTER

```
case SENSOR_FEATURE_SET_ESHUTTER:
    set_shutter(*feature_data);
    break;
```

- 每个AE周期会根据AE算法找到的对应pline table中相应index的exposure time转换为shutter下给driver.
- 长曝光的Porting请参考DCC 文档Long Exposure Porting And Issue Analysis

7.6、Feature id: SENSOR_FEATURE_SET_ESHUTTER


```
static void write_shutter(kal_uint32 shutter)
{
    kal_uint16 realtime_fps = 0;

    spin_lock(&imgsensor_drv_lock);
    if (shutter > imgsensor.min_frame_length - imgsensor_info.margin)
        imgsensor.frame_length = shutter + imgsensor_info.margin;
    else
        imgsensor.frame_length = imgsensor.min_frame_length;
    if (imgsensor.frame_length > imgsensor_info.max_frame_length)
        imgsensor.frame_length = imgsensor_info.max_frame_length;
    spin_unlock(&imgsensor_drv_lock);

    if (shutter < imgsensor_info.min_shutter)
        shutter = imgsensor_info.min_shutter;

    if (imgsensor.autoflicker_en) {
        realtime_fps = imgsensor.pclk / imgsensor.line_length * 10 / imgsensor.frame_length;
        if (realtime_fps >= 297 && realtime_fps <= 305)
            set_max_framerate(296, 0);
        else if (realtime_fps >= 147 && realtime_fps <= 150)
            set_max_framerate(146, 0);
        else {
            write_cmos_sensor_8(0x0104, 0x01);
            write_cmos_sensor_8(0x0340, imgsensor.frame_length >> 8);
            write_cmos_sensor_8(0x0341, imgsensor.frame_length & 0xFF);
            write_cmos_sensor_8(0x0104, 0x00);
        }
    } else {
        write_cmos_sensor_8(0x0104, 0x01);
        write_cmos_sensor_8(0x0340, imgsensor.frame_length >> 8);
        write_cmos_sensor_8(0x0341, imgsensor.frame_length & 0xFF);
        write_cmos_sensor_8(0x0104, 0x00);
    }
    /* Update Shutter */
    write_cmos_sensor_8(0x0104, 0x01);
    write_cmos_sensor_8(0x0202, (shutter >> 8) & 0xFF);
    write_cmos_sensor_8(0x0203, shutter  & 0xFF);
    write_cmos_sensor_8(0x0104, 0x00);
    LOG_INF("Exit! shutter =%d, framelength =%d\n", shutter, imgsensor.frame_length);
}  /*  write_shutter  */
```

根据shutter判断framelength应该写多少

对shutter作保护

Autoflicker功能开启时，
sensor需要避开30fps,15fps

Update Shutter

7.7、 Feature id: SENSOR_FEATURE_SET_GAIN

```
case SENSOR_FEATURE_SET_GAIN:
        set_gain((UINT16) *feature_data);
        break;
```

	Plinetable	driver	sensor
base	1024	64	register
Example：下2倍gain	2048	128	Mode1：gain 公式 Mode2：gain table

```
static kal_uint16 set_gain(kal_uint16 gain)
{
    kal_uint16 reg_gain, max_gain = imgsensor_info.max_gain;

    if (imgsensor.sensor_mode == IMGSENSOR_MODE_CUSTOM3) {
        /* 48M@30FPS */
        max_gain = 16 * BASEGAIN;
    }

    if (gain < imgsensor_info.min_gain || gain > max_gain) {
        pr_debug("Error gain setting");

        if (gain < imgsensor_info.min_gain)
            gain = imgsensor_info.min_gain;
        else if (gain > max_gain)
            gain = max_gain;
    }

    reg_gain = gain2reg(gain);
    spin_lock(&imgsensor_drv_lock);
    imgsensor.gain = reg_gain;
    spin_unlock(&imgsensor_drv_lock);
    pr_debug("gain = %d, reg_gain = 0x%x, max_gain:0x%x\n ",
        gain, reg_gain, max_gain);

    write_cmos_sensor_8(0x0104, 0x01);
    write_cmos_sensor_8(0x0204, (reg_gain>>8) & 0xFF);
    write_cmos_sensor_8(0x0205, reg_gain & 0xFF);
    write_cmos_sensor_8(0x0104, 0x00);

    return gain;
} /* set_gain */
```

Sensor所支持的gain范围保护

Gain倍数到register的转换

```
static kal_uint16 gain2reg(const kal_uint16 gain)
{
    kal_uint16 reg_gain = 0x0;

    reg_gain = 1024 - (1024*64)/gain;
    return (kal_uint16) reg_gain;
}
```

写gain register

7.8、 Feature id: SENSOR_FEATURE_SET_SHUTTER_FRAME_TIME

```
case SENSOR_FEATURE_SET_SHUTTER_FRAME_TIME:
    set_shutter_frame_length((UINT16) (*feature_data),
                (UINT16) (*(feature_data + 1)),
                (BOOL) (*(feature_data + 2)));
    break;
```

- 用于双摄帧同步
- 请参考DCC文档Dual_Cam_Frame_Sync_Sensor_Part_Debug_SOP

```
static void set_shutter_frame_length(kal_uint16 shutter,
                                     kal_uint16 frame_length,
                                     kal_bool auto_extend_en)
{
    unsigned long flags;
    kal_uint16 realtime_fps = 0;
    kal_int32 dummy_line = 0;

    kal_uint32 record_framelength;
    spin_lock_irqsave(&imgsensor_drv_lock, flags);
    imgsensor.shutter = shutter;
    spin_unlock_irqrestore(&imgsensor_drv_lock, flags);

    record_framelength = ((read_cmos_sensor_8(0x0341) & 0xFF)
                        | (read_cmos_sensor_8(0x0340) << 8));
    spin_lock(&imgsensor_drv_lock);
    /* Change frame time */
    if (frame_length > 1)
        dummy_line = frame_length - imgsensor.frame_length;

    imgsensor.frame_length = imgsensor.frame_length + dummy_line;

//  if (shutter > imgsensor.frame_length - imgsensor_info.margin)
//      imgsensor.frame_length = shutter + imgsensor_info.margin;

    if (imgsensor.frame_length > imgsensor_info.max_frame_length)
        imgsensor.frame_length = imgsensor_info.max_frame_length;
    spin_unlock(&imgsensor_drv_lock);
    shutter = (shutter < imgsensor_info.min_shutter)
            ? imgsensor_info.min_shutter : shutter;
    shutter =
    (shutter > (imgsensor_info.max_frame_length - imgsensor_info.margin))
        ? (imgsensor_info.max_frame_length - imgsensor_info.margin)
        : shutter;
```

判断framelength应该写多少

对shutter作保护

```
if (imgsensor.autoflicker_en) {
    realtime_fps = imgsensor.pclk / imgsensor.line_length * 10 /
            imgsensor.frame_length;
    if (realtime_fps >= 297 && realtime_fps <= 305)
        set_max_framerate(296, 0);
    else if (realtime_fps >= 147 && realtime_fps <= 150)
        set_max_framerate(146, 0);
    else {
        /* Extend frame length */
        write_cmos_sensor_8(0x0104, 0x01);
        write_cmos_sensor_8(0x0340,imgsensor.frame_length >> 8);
        write_cmos_sensor_8(0x0341,imgsensor.frame_length & 0xFF);
        write_cmos_sensor_8(0x0104, 0x00);
    }
} else {
    /* Extend frame length */
    write_cmos_sensor_8(0x0104, 0x01);
    write_cmos_sensor_8(0x0340, imgsensor.frame_length >> 8);
    write_cmos_sensor_8(0x0341, imgsensor.frame_length & 0xFF);
    write_cmos_sensor_8(0x0104, 0x00);
}
```

Autoflicker功能开启时，
sensor需要避开30fps,15fps

```
/* Update Shutter */
write_cmos_sensor_8(0x0104, 0x01);
if (auto_extend_en)
    write_cmos_sensor_8(0x0350, 0x01); /* Enable auto extend */
else
    write_cmos_sensor_8(0x0350, 0x00); /* Disable auto extend */
write_cmos_sensor_8(0x0202, (shutter >> 8) & 0xFF);
write_cmos_sensor_8(0x0203, shutter  & 0xFF);
write_cmos_sensor_8(0x0104, 0x00);
pr_debug(
    "Exit! shutter =%d, framelength =%d/%d, record_framelength = %d, dummy_line=%d, auto_extend=%d\n",
    shutter, imgsensor.frame_length, frame_length,
    record_framelength, dummy_line, read_cmos_sensor_8(0x0350));
```

Update Shutter

Sony sensor有auto extend framelength的功能

```
}   /* set_shutter_frame_length */
```

7.9、Feature id:

SENSOR_FEATURE_SET_REGISTER,

SENSOR_FEATURE_GET_REGISTER

```
case SENSOR_FEATURE_SET_REGISTER:
        write_cmos_sensor_8(sensor_reg_data->RegAddr,
                            sensor_reg_data->RegData);
        break;
case SENSOR_FEATURE_GET_REGISTER:
        sensor_reg_data->RegData =
                read_cmos_sensor_8(sensor_reg_data->RegAddr);
        break;
```

- **Debug使用的接口，** 可用adb动态读写sensor regsiter.

- 使用方法, 可参考ID: FAQ05758 **[Camera Drv]MT6589平台如何通过adb动态调试sub sensor的register**

7.10、Feature id: SENSOR_FEATURE_CHECK_SENSOR_ID

```
case SENSOR_FEATURE_CHECK_SENSOR_ID:
        get_imgsensor_id(feature_return_para_32);
        break;
```

- Search sensor时候获取Sensor id的接口

7.11、Feature id: SENSOR_FEATURE_SET_AUTO_FLICKER_MODE

- CAMERA下面， 点击Anti-flicker, 选择”auto”， 就会调用set_auto_flicker_mode函数，传true下来，记录imgsensor.autoflicker_en变量, 之后set_shutter中会用到,
- Auto flicker算法检测， 需要避开15fps,30fps帧率，所以set_shutter中会计算当前帧率，相应调整为14.6fps, 29.6fps, 否则算法检测会出错.

```
case SENSOR_FEATURE_SET_AUTO_FLICKER_MODE:
    set_auto_flicker_mode((BOOL)*feature_data_16,
                          *(feature_data_16+1));
    break;

static kal_uint32 set_auto_flicker_mode(kal_bool enable, UINT16 framerate)
{
    spin_lock(&imgsensor_drv_lock);
    if (enable) /*enable auto flicker*/ {
        imgsensor.autoflicker_en = KAL_TRUE;
        pr_debug("enable! fps = %d", framerate);
    } else {
        /*Cancel Auto flick*/
        imgsensor.autoflicker_en = KAL_FALSE;
    }
    spin_unlock(&imgsensor_drv_lock);

    return ERROR_NONE;
}
```

7.12、Feature id:

SENSOR_FEATURE_SET_MAX_FRAME_RATE_BY_SCENARIO

- 对各个SCENARIO下配置最大桢率，需要将最大桢率限制在该api带下来的桢率值， (10base) eg: 300表示30fps

```
case SENSOR_FEATURE_SET_MAX_FRAME_RATE_BY_SCENARIO:
    set_max_framerate_by_scenario(
            (enum MSDK_SCENARIO_ID_ENUM)*feature_data,
            *(feature_data+1));
    break;
```

```
static kal_uint32 set_max_framerate_by_scenario(
        enum MSDK_SCENARIO_ID_ENUM scenario_id, MUINT32 framerate)
{
    kal_uint32 frame_length;

    pr_debug("scenario_id = %d, framerate = %d\n", scenario_id, framerate);

    switch (scenario_id) {
    case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
        frame_length = imgsensor_info.pre.pclk / framerate * 10
                / imgsensor_info.pre.linelength;
        spin_lock(&imgsensor_drv_lock);
        imgsensor.dummy_line =
            (frame_length > imgsensor_info.pre.framelength)
        ? (frame_length - imgsensor_info.pre.framelength) : 0;
        imgsensor.frame_length =
            imgsensor_info.pre.framelength
            + imgsensor.dummy_line;
        imgsensor.min_frame_length = imgsensor.frame_length;
        spin_unlock(&imgsensor_drv_lock);
        if (imgsensor.frame_length > imgsensor.shutter)
            set_dummy();
        break;
```

此处set_dummy(),必须存在，否则会导致cts测试fail，详见faq：19520

调整Frame length/Line lenth

```
static void set_dummy(void)
{
    pr_debug("dummyline = %d, dummypixels = %d\n",
        imgsensor.dummy_line, imgsensor.dummy_pixel);
    /* return;*/ /* for test */
    write_cmos_sensor_8(0x0104, 0x01);

    write_cmos_sensor_8(0x0340, imgsensor.frame_length >> 8);
    write_cmos_sensor_8(0x0341, imgsensor.frame_length & 0xFF);
    write_cmos_sensor_8(0x0342, imgsensor.line_length >> 8);
    write_cmos_sensor_8(0x0343, imgsensor.line_length & 0xFF);

    write_cmos_sensor_8(0x0104, 0x00);

}   /*  set_dummy  */
```

调整VB

```
static void set_dummy(void)
{
    kal_uint32 vb = 10;
    kal_uint32 basic_line = 1224;
    vb = imgsensor.frame_length - basic_line;
    vb = (vb < 10) ? 10 : vb;
    write_cmos_sensor(0xfd, 0x01);
    write_cmos_sensor(0x05, (vb >> 8) & 0xFF);
    write_cmos_sensor(0x06, vb & 0xFF);
    write_cmos_sensor(0x01, 0x01);
}
```

7.13、Feature id: SENSOR_FEATURE_GET_DEFAULT_FRAME_RATE_BY_SCENARIO

获取各个scenario下的最大桢率（10base）,eg: 300表示30fps.

```
case SENSOR_FEATURE_GET_DEFAULT_FRAME_RATE_BY_SCENARIO:
    get_default_framerate_by_scenario(
            (enum MSDK_SCENARIO_ID_ENUM)*(feature_data),
            (MUINT32 *)(uintptr_t)(*(feature_data+1)));
    break;
```

```
static kal_uint32 get_default_framerate_by_scenario(
        enum MSDK_SCENARIO_ID_ENUM scenario_id, MUINT32 *framerate)
{
    pr_debug("scenario_id = %d\n", scenario_id);

    switch (scenario_id) {
    case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
        *framerate = imgsensor_info.pre.max_framerate;
        break;
    case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
        *framerate = imgsensor_info.normal_video.max_framerate;
        break;
    case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
        *framerate = imgsensor_info.cap.max_framerate;
        break;
    case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
        *framerate = imgsensor_info.hs_video.max_framerate;
        break;
    case MSDK_SCENARIO_ID_SLIM_VIDEO:
        *framerate = imgsensor_info.slim_video.max_framerate;
        break;
    case MSDK_SCENARIO_ID_CUSTOM1:
        *framerate = imgsensor_info.custom1.max_framerate;
        break;
    default:
        break;
    }

    return ERROR_NONE;
}
```

7.14、Feature id: SENSOR_FEATURE_SET_TEST_PATTERN

- Factory mode下camera auto test会通过该接口让sensor输出test pattern, 作CRC校验. Test pattern setting需保证sensor输出的每一桢数据相同bit都是完全一致的。

```
case SENSOR_FEATURE_SET_TEST_PATTERN:
    set_test_pattern_mode((BOOL)*feature_data);
    break;
```

Eg.

Table 17 Registers Related Test Pattern

Register Name	Index	Bit	Description
test_pattern_mode	0x0601	[2:0]	0 = No pattern (default) 1 = Solid color 2 = 100 % color bars 3 = Fade to grey' color bars 4 to 255 = Reserved

```
static kal_uint32 set_test_pattern_mode(kal_bool enable)
{
    pr_debug("enable: %d\n", enable);

    if (enable)
        write_cmos_sensor_8(0x0601, 0x0002); /*100% Color bar*/
    else
        write_cmos_sensor_8(0x0601, 0x0000); /*No pattern*/

    spin_lock(&imgsensor_drv_lock);
    imgsensor.test_pattern = enable;
    spin_unlock(&imgsensor_drv_lock);
    return ERROR_NONE;
}
```

0x02: color bar

根据datasheet 来实现

7.15、Feature id: SENSOR_FEATURE_GET_TEST_PATTERN_CHECKSUM_VALUE

供产线上camera auto检测时，系统根据抓下来的一桢test pattern数据用同样算法计算CRC，和Driver填写的CRC比对，如果一致，test pass, 反之fail, 提示硬件连线有问题.

```
case SENSOR_FEATURE_GET_TEST_PATTERN_CHECKSUM_VALUE:
    /* for factory mode auto testing */
    *feature_return_para_32 = imgsensor_info.checksum_value;
    *feature_para_len = 4;
    break;
```

该值如何填写：Factory mode下camera auto test时抓取main log，从中搜索crc关键字，各种环境下多测试几次，保证每次计算出来的值一致后，将该值记录下来，填到imgsensor_info.checksum_value中

7.16、Feature id: SENSOR_FEATURE_SET_FRAMERATE

```
case SENSOR_FEATURE_SET_FRAMERATE:
    pr_debug("current fps :%d\n", (UINT32)*feature_data_32);
    spin_lock(&imgsensor_drv_lock);
    imgsensor.current_fps = *feature_data_32;
    spin_unlock(&imgsensor_drv_lock);
    break;
```

- 记录当前的framerate(10base), eg: 300表示30fps.
- 这个feature id是在每个sensor mode切换之前带下来，表示各个mode初始最大帧率要设置为多少，会参考SENSOR_FEATURE_GET_DEFAULT_FRAME_RATE_BY_SCENARIO回传给上层的最大帧率。

7.17、Feature id: SENSOR_FEATURE_GET_CROP_INFO

此信息供LSC参考，回传各个sensor mode, sensor输出的window size相对于capture size是怎样得到的.

```
case SENSOR_FEATURE_GET_CROP_INFO:
    wininfo =
(struct SENSOR_WINSIZE_INFO_STRUCT *)(uintptr_t)(*(feature_data+1));

    switch (*feature_data_32) {
    case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[1], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[2], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[3], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_SLIM_VIDEO:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[4], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_CUSTOM1:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[5], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
    default:
        memcpy((void *)wininfo, (void *)&imgsensor_winsize_info[0], sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    }
    break;
```

Example:Sensor Crop information

- feature command : SENSOR_FEATURE_GET_CROP_INFO

```
struct SENSOR_WINSIZE_INFO_STRUCT {
    MUINT16 full_w;
    MUINT16 full_h;
    MUINT16 x0_offset;
    MUINT16 y0_offset;
    MUINT16 w0_size;
    MUINT16 h0_size;
    MUINT16 scale_w;
    MUINT16 scale_h;
    MUINT16 x1_offset;
    MUINT16 y1_offset;
    MUINT16 w1_size;
    MUINT16 h1_size;
    MUINT16 x2_tg_offset;
    MUINT16 y2_tg_offset;
    MUINT16 w2_tg_size;
    MUINT16 h2_tg_size;
};
/* Sensor output window information*/
static struct SENSOR_WINSIZE_INFO_STRUCT imgsensor_winsize_info[6] = {
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,
       24,   20, 4608, 3456,    0,    0, 4608, 3456},/* preview */
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,
       24,   20, 4608, 3456,    0,    0, 4608, 3456},/* capture */
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,
       24,  452, 4608, 2592,    0,    0, 4608, 2592},/*normal_video*/
    {4656, 3496,    0,    0, 4656, 3496, 2328, 1748,
      204,  334, 1920, 1080,    0,    0, 1920, 1080},/* hs_video */
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,
       24,  252, 4608, 2992,    0,    0, 4608, 2992},/* slim vedio */
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,
       24,   20, 4608, 3456,    0,    0, 4608, 3456},/*custom1*/
};
```

```
case SENSOR_FEATURE_GET_CROP_INFO:
    LOG_INF("SENSOR_FEATURE_GET_CROP_INFO scenarioId:%d\n",
        (UINT32) *feature_data);
    wininfo =
(struct SENSOR_WINSIZE_INFO_STRUCT *) (uintptr_t) (*(feature_data + 1));

    switch (*feature_data_32) {
    case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
        memcpy((void *)wininfo,
            (void *)&imgsensor_winsize_info[1],
            sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
        memcpy((void *)wininfo,
            (void *)&imgsensor_winsize_info[2],
            sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
    case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
        memcpy((void *)wininfo,
            (void *)&imgsensor_winsize_info[3],
            sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
        break;
```

- limitation : full size (4:3) or full size (16:9) width and height need 4x alignment

Example:Sensor Crop information

isp6s增加了对driver中所填的winsizeinfo进行了一次precheck的机制，若不满足该rule，则会CRDISPATCH_KEY:Sensor_Info_preCheck引起NE，若有Special Sensor无法满足如下Rule，可提case给MTK评估如何处理。

- **Rule #1: 所有 sensor mode 的 full size 应该相等**

m_rSensorCropWin[i].full_w != m_rSensorCropWin[m_eFullSensorMode].full_w ||
m_rSensorCropWin[i].full_h != m_rSensorCropWin[m_eFullSensorMode].full_h
(Error log 会打出有误的 Sensor mode 编号 及 full size 宽高)

- **Rule #2: check 两个 API 取出的 final size 相等**

pIHalSensorList->querySensorStaticInfo
pIHalSensor->sendCommand
以 SensorCustom1 为例
rSensorCropInfo.w2_tg_size != rSensorStaticInfo.SensorCustom1Width ||
rSensorCropInfo.h2_tg_size != rSensorStaticInfo.SensorCustom1Height
(Error log 会打出有误的 Sensor mode 编号 及 两个 API 取出的 final 宽高)

- **Rule #3: Resize 和 final size 需为偶数**

m_rSensorCropWin[i].w2_tg_size % 2 != 0 ||
m_rSensorCropWin[i].h2_tg_size % 2 != 0 ||
m_rSensorCropWin[i].scale_w % 2 != 0 ||
m_rSensorCropWin[i].scale_h % 2 != 0
(Error log 会打出有误的 Sensor mode 编号 及 resize / final size 的宽高)

- **Rule #4: Crop1 区域不能超过 Full size**

m_rSensorCropWin[i].x0_offset + m_rSensorCropWin[i].w0_size >
m_rSensorCropWin[i].full_w ||
m_rSensorCropWin[i].y0_offset + m_rSensorCropWin[i].h0_size >
m_rSensorCropWin[i].full_h
(Error log 会打出有误的 Sensor mode 编号 及 Crop1 / offset / full size 的宽高)

- **Rule #5: Crop1 要能被 Resize 整除 (Crop1 为 resize 的整数倍)**

m_rSensorCropWin[i].w0_size % m_rSensorCropWin[i].scale_w != 0 ||
m_rSensorCropWin[i].h0_size % m_rSensorCropWin[i].scale_h != 0
(Error log 会打出有误的的 Sensor mode 编号 及 Crop1 / resize 的宽高)

- **Rule #6: Resize 之后与 final output 的 ratio 应该要 <= 2**

(m_rSensorCropWin[i].scale_w / m_rSensorCropWin[i].w2_tg_size) > 2 ||
(m_rSensorCropWin[i].scale_h / m_rSensorCropWin[i].h2_tg_size) > 2
(Error log 会打出有误的 Sensor mode 编号 及 resize / final 的宽高)

- **Rule #7: 使用的 Sensor mode 不能超过 sensor mode 总数**

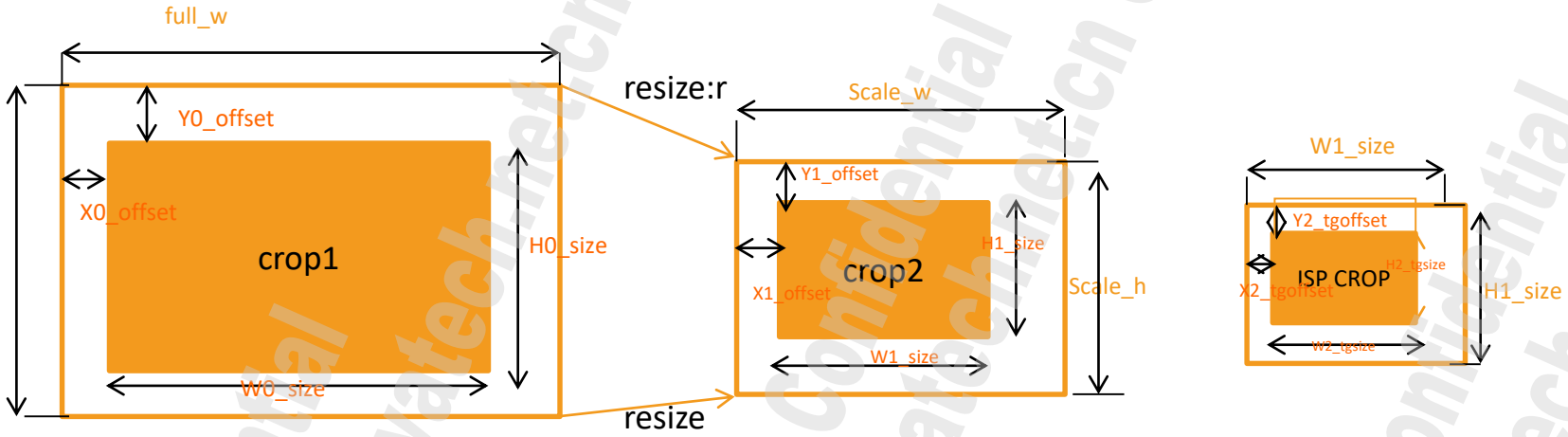
rSensorStaticInfo.SensorModeNum <= 上层设下來的sensor mode
(Error log 会打出使用到的 Sensor mode 编号 及 sensor mode 总数)

- **Note:**

从 pIHalSensor->sendCommand API 取出的值如下
Crop1 宽高 → w0_size & h0_size
Resize 宽高 → scale_w & scale_h
Final size 宽高 → w2_tg_size & h2_tg_size

Sensor Crop information

crop1 → resize(binning) →crop2 → ISP crop



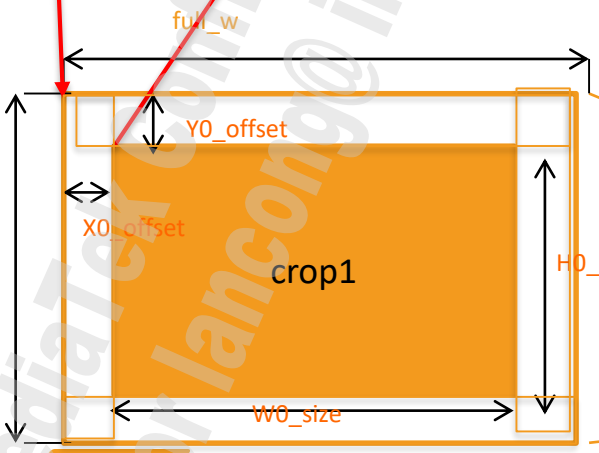
Full size: 指的是sensor output的Full size大小
Crop1: 作resizer前的从Full size得到的Crop window大小和偏移位置
resize: resizer后的宽高
Crop2: 作resizer后再次作Crop的window大小和偏移位置
（多数sensor不一定需要Crop2）
TG crop: 这个是驱动中配置的Grab window, 不需要vendor提供
每个mode都需要提供这些crop的值，为了完美实现LSC一转三，
所有preview, video mode的window要涵盖在Capture window范围内。

- Winsize Info Struct For LSC使用，该信息需要填正确，否则影响结果是可能不能预览，或者lsc效果不好等不预期的问题。

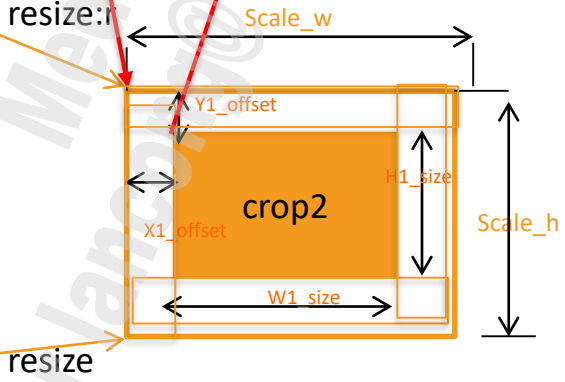
```
static struct SENSOR_WINSIZE_INFO_STRUCT imgsensor_winsize_info[6] = {
    {4656, 3496,    0,    0, 4656, 3496, 2328, 1728,    0,    0, 2328, 1728,    0,    0, 2328, 1728}, /* Preview */
    {4656, 3496,    0,    0, 4656, 3496, 4656, 3496,    0,    0, 4656, 3496,    0,    0, 4656, 3496}, /* capture */
    {4656, 3496,    0,  440, 4656, 2616, 4656, 2616,    0,    0, 4656, 2616,    0,    0, 4656, 2616}, /* video */
    {4656, 3496,  408,  668, 3840, 2160, 1920, 1080,    0,    0, 1920, 1080,    0,    0, 1920, 1080}, /* hs video */
    {4656, 3496,  408,  668, 3840, 2160, 1920, 1080,    0,    0, 1920, 1080,    0,    0, 1920, 1080}, /* slim video */
    {4656, 3496, 1048, 1028, 2560, 1440, 1280,  720,    0,    0, 1280,  720,    0,    0, 1280,  720}, /* custom1*/
```

```
struct SENSOR_WINSIZE_INFO_STRUCT {
    MUINT16 full_w;
    MUINT16 full_h;
    MUINT16 x0_offset;
    MUINT16 y0_offset;
    MUINT16 w0_size;
    MUINT16 h0_size;
    MUINT16 scale_w;
    MUINT16 scale_h;
    MUINT16 x1_offset;
    MUINT16 y1_offset;
    MUINT16 w1_size;
    MUINT16 h1_size;
    MUINT16 x2_tg_offset;
    MUINT16 y2_tg_offset;
    MUINT16 w2_tg_size;
    MUINT16 h2_tg_size;
};
```

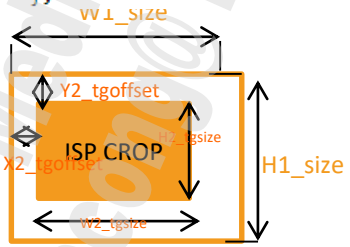
Binning mode/Sub-sampling mode



crop1: x0,y0,w0,h0

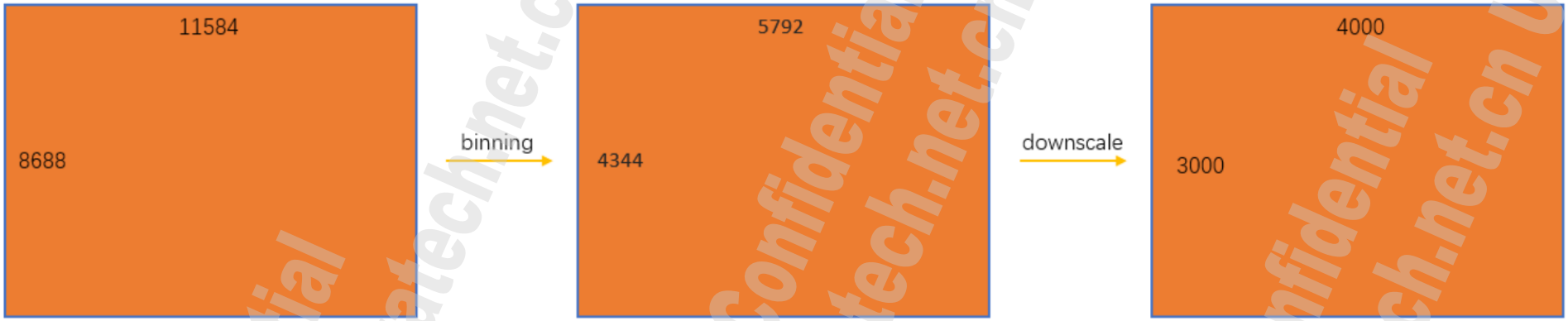


crop2: x1,y1,w1,h1



Isp crop: x2,y2,w2,h2

Special Case



SensorMode(0), full_0(11584,8688) crop1(0,0,11584,8688) resize(4000,3000) crop2(0,0,4000,3000) final size(4000,3000)

这种Special Sensor在sensor内部先binning然后再downscale，无法满足Rule 5中所述的resize需要是整数倍，这种Case MTK也可Support，可以正常填写winsizeinfo，并手动在LscMgrDefault.cpp中bypass Rule 5防止触发NE。

Driver Functions

8. get_resolution函数

- 用来配置各个mode的TG grab_window.

```
static kal_uint32
get_resolution(MSDK_SENSOR_RESOLUTION_INFO_STRUCT *sensor_resolution)
{
    pr_debug("E\n");
    sensor_resolution->SensorFullWidth = imgsensor_info.cap.grabwindow_width;
    sensor_resolution->SensorFullHeight = imgsensor_info.cap.grabwindow_height;

    sensor_resolution->SensorPreviewWidth = imgsensor_info.pre.grabwindow_width;
    sensor_resolution->SensorPreviewHeight = imgsensor_info.pre.grabwindow_height;

    sensor_resolution->SensorVideoWidth = imgsensor_info.normal_video.grabwindow_width;
    sensor_resolution->SensorVideoHeight = imgsensor_info.normal_video.grabwindow_height;

    sensor_resolution->SensorHighSpeedVideoWidth = imgsensor_info.hs_video.grabwindow_width;
    sensor_resolution->SensorHighSpeedVideoHeight = imgsensor_info.hs_video.grabwindow_height;

    sensor_resolution->SensorSlimVideoWidth = imgsensor_info.slim_video.grabwindow_width;
    sensor_resolution->SensorSlimVideoHeight = imgsensor_info.slim_video.grabwindow_height;
    return ERROR_NONE;
} /* get_resolution */
```

mode grab windows 说明

目的：在sensor输出的window上设置TG抓取的window宽高是多少，

要求：

1.设置的宽必须是**16**的倍数，高必须为4的倍数。

2. 相同比例下视角一致；4:3与16:9要求sensor输出的window保证水平方向和full size视角一致。

3.grab window设置建议和sensor输出的window一致，

特殊情况下是: grabwindow_width(height)<=sensor output width(height)-startx(y)

Driver Functions

9. control函数

- 设置各scenario对应的sensor mode, (若有需求，请添加客制化的custom mode)

```
static kal_uint32 control(enum MSDK_SCENARIO_ID_ENUM scenario_id,
                          MSDK_SENSOR_EXPOSURE_WINDOW_STRUCT *image_window,
                          MSDK_SENSOR_CONFIG_STRUCT *sensor_config_data)
{
    pr_debug("scenario_id = %d\n", scenario_id);
    spin_lock(&imgsensor_drv_lock);
    imgsensor.current_scenario_id = scenario_id;
    spin_unlock(&imgsensor_drv_lock);
    switch (scenario_id) {
    case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
        preview(image_window, sensor_config_data);
        break;
    case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
        capture(image_window, sensor_config_data);
        break;
    case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
        normal_video(image_window, sensor_config_data);
        break;
    case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
        hs_video(image_window, sensor_config_data);
        break;
    case MSDK_SCENARIO_ID_SLIM_VIDEO:
        slim_video(image_window, sensor_config_data);
        break;
    case MSDK_SCENARIO_ID_CUSTOM1:
        custom1(image_window, sensor_config_data);
        break;
    default:
        pr_debug("Error ScenarioId setting");
        preview(image_window, sensor_config_data);
        return ERROR_INVALID_SCENARIO_ID;
    }
    return ERROR_NONE;
}   /* control() */
```

Sensor_mode设置

```
static kal_uint32 hs_video(MSDK_SENSOR_EXPOSURE_WINDOW_STRUCT *image_window,
                           MSDK_SENSOR_CONFIG_STRUCT *sensor_config_data)
{
    pr_debug("E\n");

    spin_lock(&imgsensor_drv_lock);
    imgsensor.sensor_mode = IMGSENSOR_MODE_HIGH_SPEED_VIDEO;
    imgsensor.pclk = imgsensor_info.hs_video.pclk;
    /*imgsensor.video_mode = KAL_TRUE;*/
    imgsensor.line_length = imgsensor_info.hs_video.linelength;
    imgsensor.frame_length = imgsensor_info.hs_video.framelength;
    imgsensor.min_frame_length = imgsensor_info.hs_video.framelength;
    imgsensor.dummy_line = 0;
    imgsensor.dummy_pixel = 0;
    /*imgsensor.current_fps = 300;*/
    imgsensor.autoflicker_en = KAL_FALSE;
    spin_unlock(&imgsensor_drv_lock);
    hs_video_setting();
    set_mirror_flip(imgsensor.mirror);

    return ERROR_NONE;
}    /*  hs_video   */
```

设置imgsensor结构体

↓

更新寄存器

↓

设置mirror

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

I2C不通

- 对照spec检查上电时序（AVDD, DVDD, DOVDD，RST，PowerDown)；
- 对照上电，用示波器观察信号波形（SDA/SCL);
- I2C slave address 是否设置正确；
- 检查I2C 设备HW layout是否挂载当前使用的I2C bus上面；
- 确认I2C的读写函数的addr和data的数据长度，要与sensor的I2C接口一致。
- 检查cfg_setting_imgsensor.cpp 中函数getSensorMclkConnection,是否和HW上实际MCLK连接情况一致。

工厂模式进入后摄很慢

- Root Cause:

  在Search Sensor的时候，出现很多次I2C Timeout，所导致进入后摄很慢；log如下图：

```
factory][mt-i2c]ERROR,481: id=0,addr: 10, transfer timeout
factory][mt-i2c]ERROR,481: id=0,addr: 10, transfer timeout
```

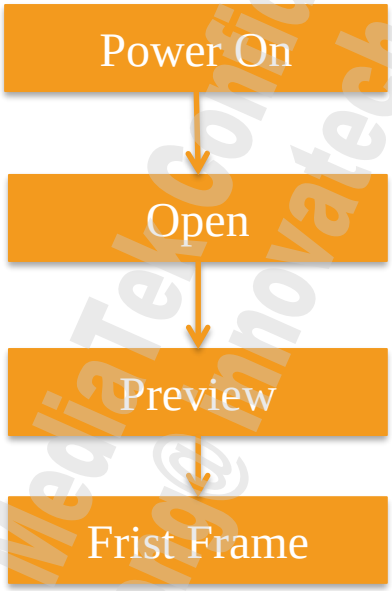
检查所有兼容sensor的上电时序，特别是Sensor PowerDown PIN 和 Reset PIN 的 Active Level;

TG获取数据失败

- 若发生TG获取数据失败，您看到的现象可能红屏，也可能是黑屏，可以参考如下方法debug：
  1. 使用示波器去测量 MIPI Data Lane是否有数据，如果没有，则看驱动的stream on/off是否打开，如果已经打开，则联系vendor解决；
  2. 检查HW layout 的中mipi port与cfg_setting_imgsensor.cpp 文件中gCustomCfg.port配置是否一致，如果不一致，则需按照HW layout修改；

Camera启动时间过长--驱动分析篇

1. 从点击Camera图标到第一帧图传出来，Driver这边flow如下：



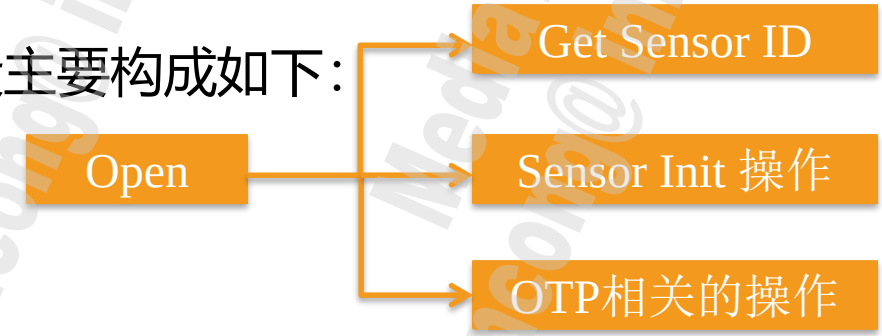
每一个阶段详细分析如下：

Camera启动时间过长--驱动分析篇

2. Power On阶段的时间

主要耗在Sensor上电时序要求的延时上面，但是如果上电code写的不规范，delay time并没有完全按照spec来实现，就会导致只是上电就用电上百毫秒，所以在分析Camera 启动时间过长的时候，请首先确定Power On 阶段耗时多少。

Open阶段主要构成如下：



Camera启动时间过长--驱动分析篇

2.1 Get Sensor ID阶段

如果模组的地址是确认，请去掉不相关地址，不要写多余的地址，减少try 地址的时间；

2.2 Sensor Init操作阶段

如果Sensor Init操作消耗了比较多时间，可以提高I2C Speed看时间是否有改善，如果没有多少的改善，可以采用i2c burst write mode来实现,但是如果寄存器不多的话，这种方式改善不大；

2.3 OTP 相关操作阶段

改善方面同Sensor Init一样，采用i2c burst read mode，如果是连续地址，请采用i2c burst read mode方式；这样改善空间比较大；

Camera启动时间过长--驱动分析篇

3. Preview阶段，可以分为如下两个阶段：

3.1 Stream Off/On操作

一般需要延迟一段时间，但是如果驱动写的不合理，或者调试驱动的时候加长delay时间而忘记修改回来，遇到这种delay时间比较长的问题，请与vendor确认delay时间的准确性；

3.2 Preview Init操作

消耗时间的改善方法，和sensor init操作一样；

Camera启动时间过长--驱动分析篇

4 First Frame阶段，

如果驱动里面pre_delay_frame/cap_delay_frame/

Video_delay_frame/hs_video_delay_frame/slim_video_delay_frame值过大，

这里可以和vendor商量，看有无调整空间。

该值在ISP4.0之后不再使用

摄像头MCLK修改为26M

- Root Cause:
  为了规避射频干扰，驱动文件里调整了MCLK及MIPI CLK，还需要根据驱动里面的修改做如下调整：
  - 更新deviceinfo;
  - 根据上一步更新的dev iceinfo更新Aepline和camera_tuning_para文件；
  - 重新生成Flicker文件，方法： FAQ：FAQ08423

Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity、OB)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

1、Sentest

Refrence FAQ22328

- Read sensor id test

若遇到camera点不亮、某个mode黑屏、卡死等状况，在提交E-service之前可先进行如下Check来确认sensor driver端是否是正常的：

adb root

adb shell sentest

会打印出如下所示search到的所有sensor,

若提示找不到sentest:

make sentest

adb push /libcameracustom.so /vendor/lib64

```
$ adb shell sentest
[main]sizeof long : 8
[show_Sensors]sensorNum 5
[show_Sensors]name:SENSOR_DRVNAME_[redacted] type:0
[show_Sensors]index:0, SensorDevIdx:1
[show_Sensors]name:SENSOR_DRVNAME_[redacted] type:0
[show_Sensors]index:1, SensorDevIdx:2
[show_Sensors]name:SENSOR_DRVNAME_[redacted] type:0
[show_Sensors]index:2, SensorDevIdx:4
[show_Sensors]name:SENSOR_DRVNAME_[redacted] type:0
[show_Sensors]index:3, SensorDevIdx:8
[show_Sensors]name:SENSOR_DRVNAME_[redacted] type:0
[show_Sensors]index:4, SensorDevIdx:16
[main]Param: 1 <sensorDev> <scenario> <fps>
[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)
[main]<scenario>  : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)
```

1、Sentest

- MIPI pixel rate check

若某颗sensor可以读到sensor id但是无法点亮，可按照如下方式check 来确认sensor driver是否有正常porting来撇清是否是sensor driver配置的问题：

- adb shell sentest [sensorDev] [scenario]
  - sensorDev - 1:main, 2:sub, 4:main2, ……
  - Scenario - 0:preview, 1:capture, 2: normal_video, ……
  - E.X. adb shell sentest 1 0

    => main cam+preview mode

```
[test_SensorInterface]seninf_mux1 horizontal valid count 0x4f5, horzontal blanking count 0xfd
[test_SensorInterface]blanking valid count 0x29784a, blanking blanking count 0x76ba07, fps =30

[test_SensorInterface]mipi_pixel_rate = 577.797547Mpps
[test_SensorInterface]vertical_blanking = 24.704375ms
[test_SensorInterface]horizontal_blanking = 0.000803ms
[test_SensorInterface]line time = 0.004832ms
```

- Sensor driver:
  - E.X. imx519mipiraw_Sensor.c

    (kernel-4.14\drivers\misc\mediatek\imgsensor\src\common\v1_1\imx519_mipi_raw)

1、Sentest

- Vertical blanking check

- adb shell sentest [sensorDev] [scenario]
  - sensorDev - 1:main, 2:sub, 4:main2, ……
  - Scenario - 0:preview, 1:capture, 2: normal_video, ……
  - E.X. adb shell sentest 1 0

    => main cam+preview mode

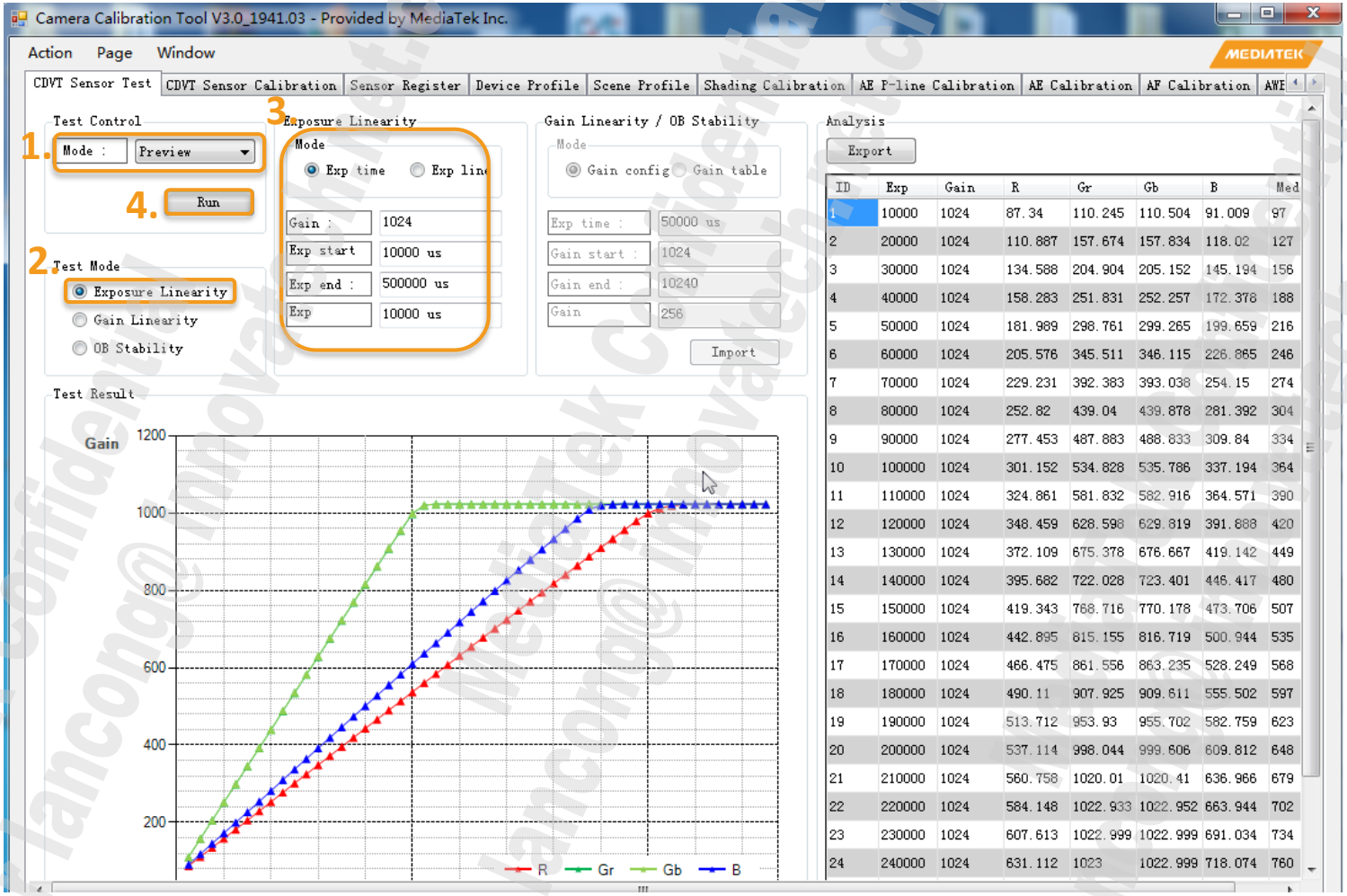
```
[test_SensorInterface]seninf_mux1 horizontal valid count 0x4f5, horzontal blanking count 0xfd
[test_SensorInterface]blanking valid count 0x29784a, blanking blanking count 0x76ba07, fps =30

[test_SensorInterface]mipi_pixel_rate = 577.797547Mpps
[test_SensorInterface]vertical_blanking = 24.704375ms
[test_SensorInterface]horizontal_blanking = 0.000803ms
[test_SensorInterface]line time = 0.004832ms
```

2、CDVT

- CCT Exposure Linearity

  Refrence guide： Raw sensor Function Test Guide Line V1.5.pptx

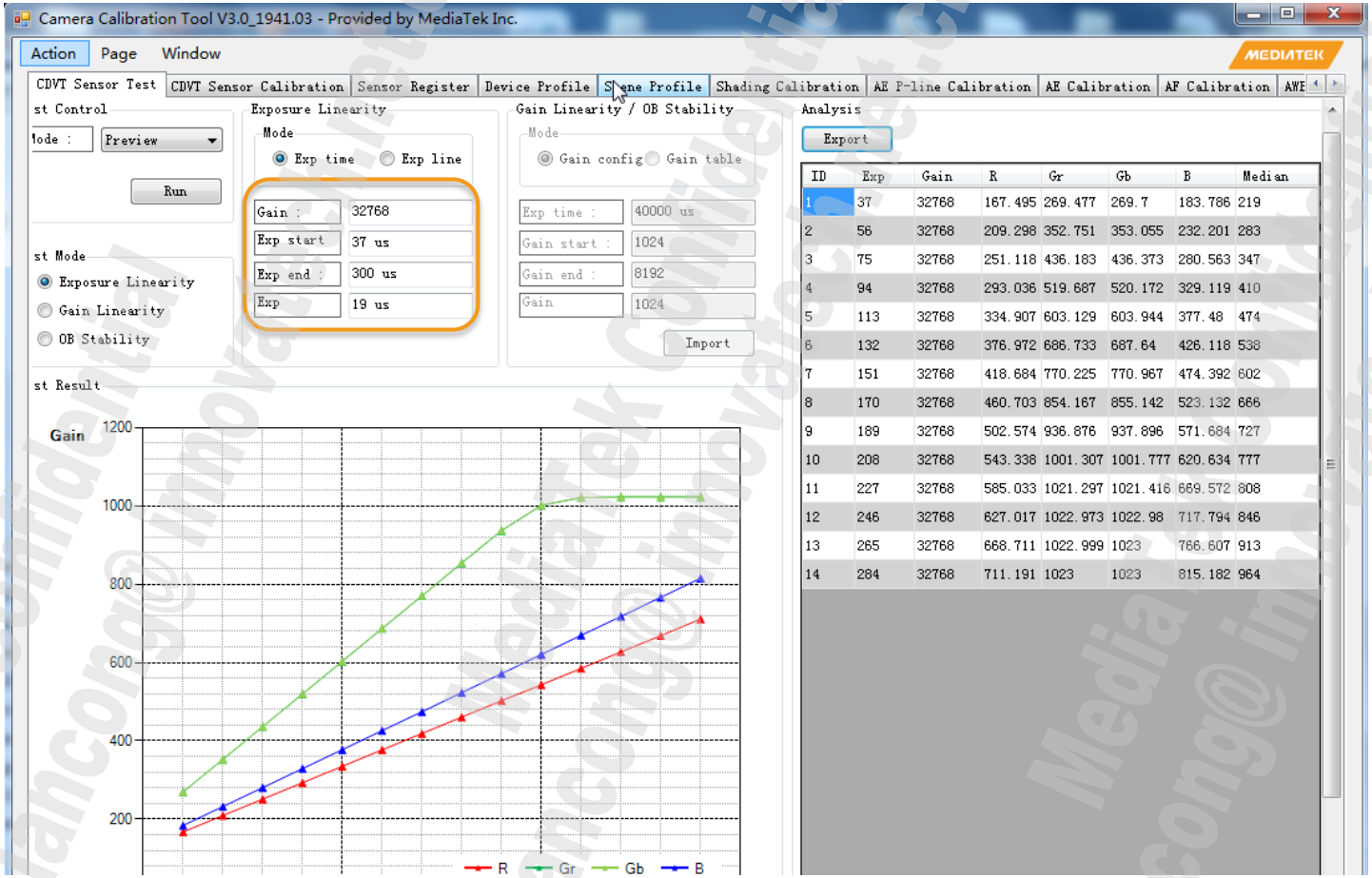


ID	Exp	Gain	R	Gr	Gb	B	Med
1	10000	1024	87.34	110.245	110.504	91.009	97
2	20000	1024	110.887	157.674	157.834	118.02	127
3	30000	1024	134.588	204.904	205.152	145.194	156
4	40000	1024	158.283	251.831	252.257	172.378	188
5	50000	1024	181.989	298.761	299.265	199.659	216
6	60000	1024	205.576	345.511	346.115	226.865	246
7	70000	1024	229.231	392.383	393.038	254.15	274
8	80000	1024	252.82	439.04	439.878	281.392	304
9	90000	1024	277.453	487.883	488.833	309.84	334
10	100000	1024	301.152	534.828	535.786	337.194	364
11	110000	1024	324.861	581.832	582.916	364.571	390
12	120000	1024	348.459	628.598	629.819	391.888	420
13	130000	1024	372.109	675.378	676.667	419.142	449
14	140000	1024	395.682	722.028	723.401	446.417	480
15	150000	1024	419.343	768.716	770.178	473.706	507
16	160000	1024	442.895	815.155	816.719	500.944	535
17	170000	1024	466.475	861.556	863.235	528.249	568
18	180000	1024	490.11	907.925	909.611	555.502	597
19	190000	1024	513.712	953.93	955.702	582.759	623
20	200000	1024	537.114	998.044	999.606	609.812	648
21	210000	1024	560.758	1020.01	1020.41	636.966	679
22	220000	1024	584.148	1022.933	1022.952	663.944	702
23	230000	1024	607.613	1022.999	1022.999	691.034	734
24	240000	1024	631.112	1023	1022.999	718.074	760

2、CDVT

- CCT Exposure Linearity

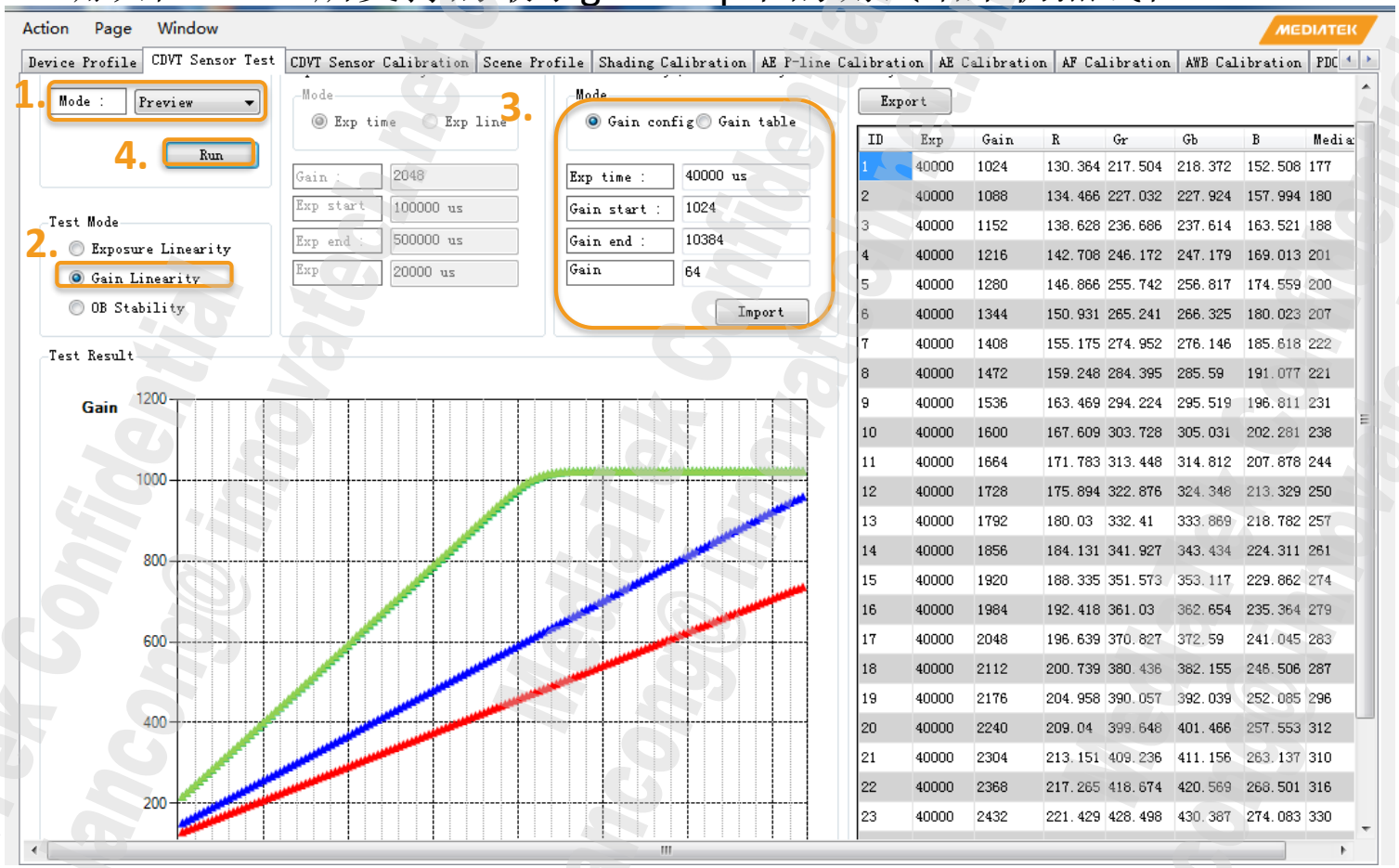
确认在sensor所支持的最小曝光行数为step的测试结果仍然线性



2、CDVT

- CCT Gain Linearity

确认在sensor所支持的最小gain step下的测试结果仍然线性



Agenda

- What's Changed
- Sensor Driver Architecture
- Sensor Driver Porting
  - How To Add A New Sensor
- Sensor Driver Introduction
  - Driver Data Structure Introduction
  - Sensor Mode Introduction
  - Driver Functions
- Case Study
- Pre-Check
  - Sentest
  - CDVT(Shutter&Gain Linearity)
- Appendix
  - ISP Spec
  - Drivers, tuning parameters, metadata referenced by each platform

1、ISP Spec

ISP version	Chip	ISP clock	Resolution
ISP6s	MT6883 MT6885 MT6889	624/499/392/312 MHz	64M@30fps single cam 32M+16M dual cam
			80M@24fps single cam 32M+16M dual cam
			80M@24fps single cam 32M+16M dual cam
	MT6873	624/499/392/273 MHz	64MP@30fps 32M+16M@30fps
	MT6853	624/499/392/273 MHz	48MP@30fps 20M+16M@30fps
	MT6833	624/546/392/286 MHz	48MP@30fps 16M+16M@30fps
ISP6.0	MT6779	560/416/315 MHz	32M@30fps 24M+16M HW dual cam 16M@120fps Capture

1、ISP Spec

ISP version	Chip	ISP clock	Resolution
ISP5.0	MT6771	546/364 MHZ	32MP@30fps 20MP+16MP HW dual cam
	MT6775	546/364/320 MHZ	
	MT6785	560/416/315 MHz	32M@30fps, 4-cell 48M@30fps Dual cam [26M+48M 4-cell]@30fps Tri cam [16M+16M+8M]@30fps

1、ISP Spec

ISP version	Chip	ISP clock	Resolution
ISP4.0	MT6762 MT6765	457/312/228 MHz	25MP @ 30fps 13MP + 13MP @ 30fps
	MT6768 MT6767	546/312/228 MHz	25MP @ 30fps 16MP + 16MP @ 30fps
			25MP @ 30fps 13MP + 13MP @ 30fps
	MT6797	320 MHz(LPM) 450 MHz(HPM)	32MP @ 24fps / 25MP @ 30fps
	MT6799	E1:480/364/273(252) MHz E2:504/364/504 MHz	28MP@30fps
	MT6757	315 MHz(LPM) 400 MHz(HPM)	24MP @ 24fps 13MP + 13MP dual camera
	MT6763	450/300 MHz	

1、ISP Spec

ISP version	Chip	ISP clock	
ISP3.0	MT6761	436/315 MHz	
	MT6739	300/180 MHz	
	MT6595	400 MHz(OD)	
	MT6795	317 MHz(DVFS OFF)	400 MHz(DVFS ON)
	MT6752	364 MHz	
	MT6735	166 MHz (LPM)	300 MHz (HPM)
	MT6755	286 MHz (LPM)	364 MHz (HPM)
	MT6753	166 MHz (LPM)	300 MHz (HPM)
	MT6737T	166 MHz (LPM)	300 MHz (HPM)
	MT6750 MT6738	286 MHz (LPM)	364 MHz (HPM)

1、ISP Spec

ISP version	Chip	ISP clock
ISP2.0	MT6571	260M
	MT6572	260M
	MT6589	280M
	MT6582	294M
	MT6592	294M 400M(OD)
	MT6580	294M
	MT6735M MT6735P	180M(LPM) 300M(HPM)
	MT6737M	182M(LPM) 380M(HPM)
	MT6737	182M(LPM) 442M(HPM)
	MT6570	
	MT6571	260M

2、Reference Driver, Tuning Parameter, Metadata

Chip	Main	Sub	Main2
MT6739	OV13870	OV8856	
MT6761	IMX499/S5K3L8	S5K2T7SP	S5K5E8YX
MT6765	IMX230/S5K2P7	S5K2T7SP	OV8856
MT6763	IMX338	OV8858	
MT6768	IMX519/S5K2P7	S5K2T7SP	OV8856
MT6771	IMX338/IMX386	S5K4E6	IMX386
MT6779	IMX398/IMX519	S5K2T7SP	IMX350
MT6785	IMX398/IMX519	S5K2T7SP	IMX350
MT6885	IMX586	IMX576	S5K3M5SX/IMX481
MT6873			
MT6853			
MT6833			

Metadata 附录

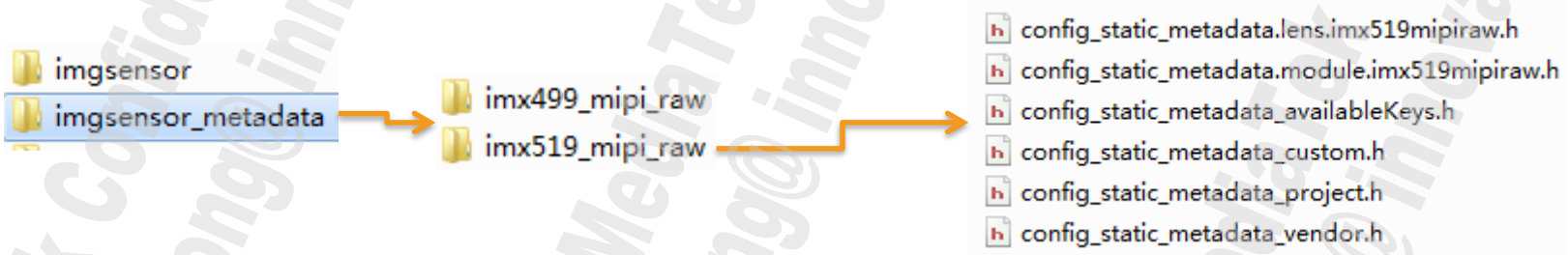
How To Add A New Sensor

- Step10 Add Sensor metadata

a) P版本之后都会走HAL3，需要配置metadata，相关文件路径如下：

/vendor/mediatek/proprietary/custom/common/hal/imgsensor_metadata/sensor/xxx_mipi_raw/

- /vendor/mediatek/proprietary/custom/ ${platform}/hal/imgsensor_metadata/xxx_mipi_raw/



b) 这里是用platform下code为例,注意若project下有对应code的话,优先级从高到低为Project>Platform>Common

How To Add A New Sensor

c) 若添加新的sensor,对应metadata添加方法如下：

- 先把codebase 中自带的imx519的 metadata 文件夹复制一份为新加的sensor 名字,并修改其文件夹内文件名,文件内sensor 名字.
- 若用**hal1.0**,以下步骤**不用再看**;若用**hal3**,按照后继步骤修改.

How To Add A New Sensor

d) /vendor/mediatek/proprietary/custom/${platform}/hal/imgsensor_metadata/common/config_static_metadata_common.h

- 若要修改为hal3.2, 目前只有6755/6797/6757支援FULL,其他仅支持limit mode.要做如下设定：

```
//=============================================================================
CONFIG_METADATA_BEGIN(MTK_INFO_SUPPORTED_HARDWARE_LEVEL)
    CONFIG_ENTRY_VALUE(MTK_INFO_SUPPORTED_HARDWARE_LEVEL_LIMITED, MUINT8)
CONFIG_METADATA_END()
//=============================================================================
```

- 若要从hal3.2 -> hal1.0. 这里不用动.

How To Add A New Sensor

e) config_static_metadata_project.h

```
//-------------------------------------------------------------------------
CONFIG_METADATA_BEGIN(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS)
    CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_BLOB, MINT64) //16mp 4:3
    CONFIG_ENTRY_VALUE(4640, MINT64)          // width
    CONFIG_ENTRY_VALUE(3480, MINT64)          // height
    CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_O
    CONFIG_ENTRY_VALUE(66666666, MINT64)      // frame duration
    CONFIG_ENTRY_VALUE(33333333, MINT64)      // stall duration

    CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_RAW16, MINT64) //16mp 4:3
    CONFIG_ENTRY_VALUE(4656, MINT64)          // width
    CONFIG_ENTRY_VALUE(3496, MINT64)          // height
    CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_O
    CONFIG_ENTRY_VALUE(33333333, MINT64)    // frame duration
    CONFIG_ENTRY_VALUE(33333333, MINT64)          // stall duratio

    CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_YCbCr_420_888, MINT64) //16
    CONFIG_ENTRY_VALUE(4656, MINT64)          // width
    CONFIG_ENTRY_VALUE(3496, MINT64)          // height
    CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_O
    CONFIG_ENTRY_VALUE(100000000, MINT64)   // frame duration
    CONFIG_ENTRY_VALUE(0, MINT64)             // stall duration

    CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_YCbCr_420_888, MINT64) //16mp 4:3
    CONFIG_ENTRY_VALUE(4640, MINT64)          // width
    CONFIG_ENTRY_VALUE(3480, MINT64)          // height
    CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64)   // output
    CONFIG_ENTRY_VALUE(100000000, MINT64)   // frame duration
    CONFIG_ENTRY_VALUE(0, MINT64)             // stall duration
CONFIG_METADATA_END()
```

Format:

1. HAL_PIXEL_FORMAT_BLOB
2. HAL_PIXEL_FORMAT_YCbCr_420_888
3. HAL_PIXEL_FORMAT_RAW16

Resolution:

1. 配置的size不要超过平台最大支持size.
2. 每组设定要16X整数倍

How To Add A New Sensor

f) config_static_metadata.sensor.imx519mipiraw.h

- 配置如下信息
  - Sensor的物理size:

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_PHYSICAL_SIZE) // mm
    CONFIG_ENTRY_VALUE(5.98f, MFLOAT)
    CONFIG_ENTRY_VALUE(4.49f, MFLOAT)
CONFIG_METADATA_END()
```

  - 根据sensor full size设定如下para.(mt6797 后该设定移入sensor driver中)

```
.startx = 4,
.starty = 4,
.grabwindow_width = 4192,
.grabwindow_height = 3104,
```

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_ACTIVE_ARRAY_REGION)
    CONFIG_ENTRY_VALUE(MRect(MPoint(4, 4), MSize(4192, 3104)), MRect)
CONFIG_METADATA_END()
```

How To Add A New Sensor

f) config_static_metadata.sensor.imx519mipiraw.h

配置如下信息

- 当前项目支持的gain的范围通常支持范围，100表示1x Gain

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_SENSITIVITY_RANGE)
    CONFIG_ENTRY_VALUE(100, MINT32)
    CONFIG_ENTRY_VALUE(6400, MINT32)
CONFIG_METADATA_END()
```

- 支持的曝光时间范围，单位ns，最小值低于100us

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_EXPOSURE_TIME_RANGE)// 1 us - 30 sec
    CONFIG_ENTRY_VALUE(100000L, MINT64)
    CONFIG_ENTRY_VALUE(16000000000L, MINT64)
CONFIG_METADATA_END()
```

How To Add A New Sensor

f) config_static_metadata.sensor.imx519mipiraw.h

配置如下信息

- sensor支持的最小帧率兑换为对应的时间值，这个值必须大于等于MTK_SENSOR_INFO_EXPOSURE_TIME_RANGE的值

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_MAX_FRAME_DURATION)// 30 sec
    CONFIG_ENTRY_VALUE(16000000000L, MINT64)
CONFIG_METADATA_END()
```

- sensor 支持的最大Analog Gain

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_MAX_ANALOG_SENSITIVITY)
    CONFIG_ENTRY_VALUE(1600, MINT32)
CONFIG_METADATA_END()
```